education

Индексация и булев поиск

«Информационный поиск и ранжирование»
Преподаватель Руслан Аллаяров

Ведущий лекции



- Более 7 лет в ML, 6 лет из них в поиске
- Ex VK (teamlead поиска), Ex OKKO (teamlead рекома)
- Отвечал за блендинг поиска по видео, за базовое качество поиска по клипам и плейлистам
- Закончил мехмат МГУ и Техносферу (2021)

education

План лекции

1. Термины, определения
2. Булев поиск
3. Обратный индекс
4. Препроцессинг документов
5. Структура обратного индекса
6. Построение больших индексов
7. Сжатие индекса
8. Оценка качества булевой модели
9. Домашнее задание

Что мы уже знаем из лекции 1

- На вводной лекции мы уже бегло прошли обратный индекс, булев поиск и метрики качества — но в контексте истории инфопоиска
- Сегодня возвращаемся к тем же темам по-настоящему: с матрицей «термин — документ», структурами данных, оценками памяти и кодом
- Слайды, которые повторяют материал лекции 1, помечены плашкой «Повторение из лекции 1» — по ним можно двигаться быстро
- Всё остальное — новое: препроцессинг документов, структуры данных индекса, построение больших индексов на кластере, сжатие и оценка качества

Информационный поиск, модели поиска

- Что такое информационный поиск?
- Информационный поиск (ИП) — это процесс поиска в большой коллекции (хранящейся, как правило, в памяти компьютера) некоего неструктурированного материала (обычно документа), удовлетворяющего информационные потребности (Manning, 2011)
- Булева модель (Boolean Retrieval Model, BIR)
- Модели поиска с ранжированием (Ranked Retrieval Models)

Повторение из лекции 1

Документы

- Будем рассматривать только текстовые документы
- Коллекция документов (или текстовый корпус, особ. в лингвистике)
  - У каждого документа уникальный DocID из [1, ...)
  - Может быть большой: >10 млрд. документов весь Рунет

Поисковые запросы

- Пользователь выражает свою информационную потребность в виде поискового запроса
- Средняя длина поискового запроса ~**2.5 слова**
- Запросы могут:
  - Быть неоднозначны (напр. *машина*) и не-грамматичны (напр. *вконтакте вход моя страница*)
  - Часто содержат ошибки и опечатки (*однакласники*)

Слова

- Документы и запросы состоят из слов
- Но со словами есть проблемы
- Одно и то же слово может быть написано по-разному: МГУ и мгу
- Слова имеют формы и окончания
- По запросу университеты мы должны находить документы со словом университет
- Понятие слова довольно неопределенно
  - C++ – это слово?
- У слова может быть несколько значений
  - Омонимы: машина

Термины

- **Термины** (terms) — базовые словоподобные единицы, на которые мы разбиваем документы и запросы
- Термины можно понимать как нормализованные слова: Москва → москва, университеты → университет, МГУ → мгу
  - Нижний регистр
  - Именительный падеж, единственное число и т.п.
  - Подробности позже
- Документы и запросы – последовательность терминов
  - Московский государственный университет → [московский, государственный, университет]

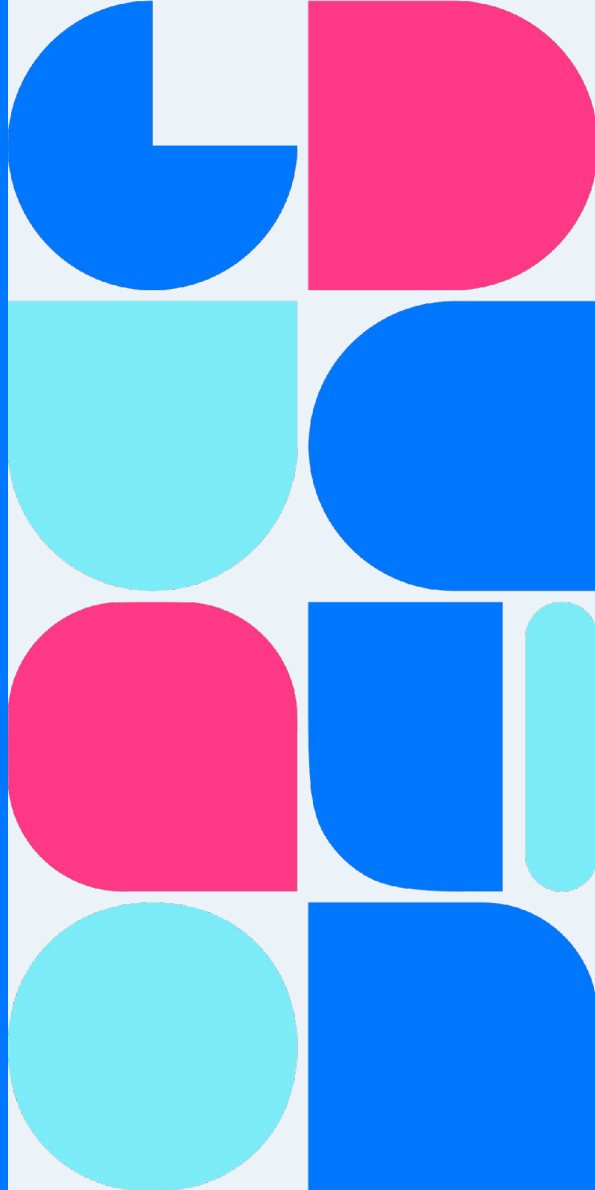
Вопросы?



education

Булев поиск

Понять, как булева модель описывает поисковые запросы и как по ним находить подходящие документы



Булева модель

- Моделируем запросы с помощью логических операций (кроме терминов добавляем операции):
  - конъюнкция (И): A && B
  - дизъюнкция (ИЛИ): A || B
  - отрицание (НЕ): ~A
- НЕ-запросы (отрицание)
  - Классическая булева модель допускает возможность использования оператора НЕ
  - Редко используется в современных поисковых системах т.к. сложно реализовать эффективно
  - В дальнейшем НЕ-запросы мы рассматривать НЕ будем

Повторение из лекции 1

И-запросы (конъюнкция терминов)

- Многословный запрос – это просто конъюнкция нескольких терминов
  - Московский государственный университет → московский && государственный && университет
- Такие запросы очень просто интерпретировать:
  - Документ должен содержать ВСЕ термины: И московский, И государственный, И университет

Повторение из лекции 1

ИЛИ-запросы (дизъюнкция терминов)

- По запросу университет мы хотим находить документы со словом *институт*
- Достигается с помощью расширения запроса синонимами
- Например:
  - Синонимы: {*университет*, *институт*}
  - *университет итмо* → (*университет* || *институт*) && *итмо*
- Либо пользователь может вручную указать “ИЛИ” в запросе, например: *кеды* || *кроссовки*
- Дизъюнкцию тоже просто интерпретировать:
  - Документ должен содержать ХОТЯ БЫ ОДИН из терминов: ИЛИ *университет*, ИЛИ *институт*

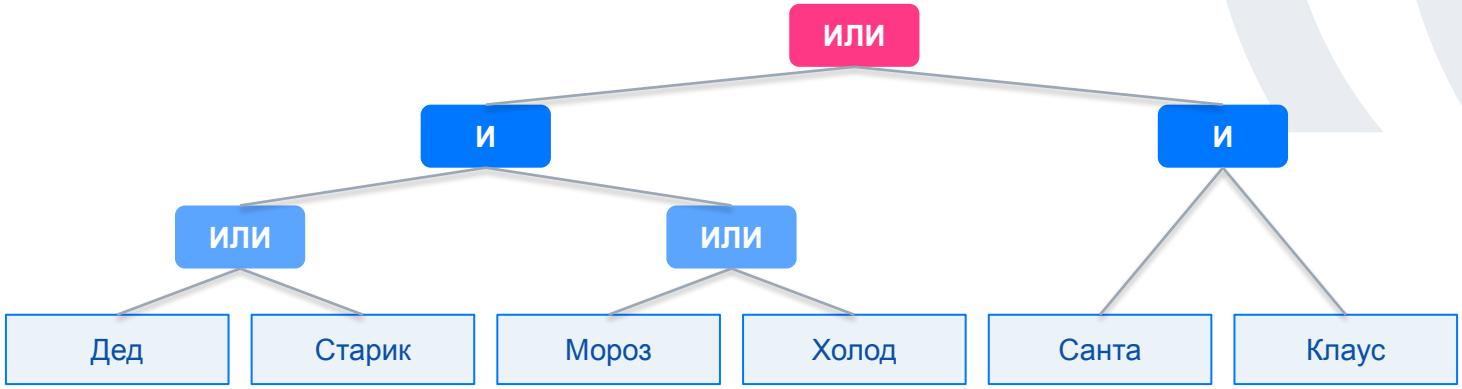
Повторение из лекции 1

Многословные синонимы

- Фраза из N слов может быть синонимом другой фразы из M слов, при этом отдельные слова могут не являться синонимами друг друга
- Примеры:
  - {дед мороз, санта клаус}
  - {япония, страна восходящего солнца}
- Запросы с многословными синонимами также представляются с помощью операторов ИЛИ и И
  - дед мороз → (дед && мороз) || (санта && клаус)

Дерево запроса

- В общем случае запрос представим в виде дерева: узлы — операции, листья — термины
- Синонимы: {дед мороз, санта клаус}, {дед, старик}, {мороз, холод}



дед мороз → ((дед || старик) && (мороз || холод)) || (санта && клаус)

Повторение из лекции 1

Тестовая коллекция документов

- Будем работать с 3 документами:
  1) Московский физико-технический институт
  2) Московский государственный университет
  3) Университет ИТМО
- Разобъем на термины:
  1) [московский, физика, технический, институт]
  2) [московский, государственный, университет]
  3) [университет, итмо]

Повторение из лекции 1

Матрица термин-документ

- Представим коллекцию документов в виде матрицы

	№1 МФТИ	№2 МГУ	№3 ИТМО
государственный	0	1	0
институт	1	0	0
итмо	0	0	1
московский	1	1	0
технический	1	0	0
физика	1	0	0
университет	0	1	1

M i j = 1, если в j-м документе есть i-е слово

Обработка И-запросов

- С помощью матрицы термин-документ легко найти множество документов, удовлетворяющих запросу-конъюнкции
- Запрос: *московский* && *университет*
- Применяем логическую операцию И к строкам
  - *московский* → 110
  - *Университет* → 011 &&
  - <И> = 010
- 010 соответствует 2-му столбцу: №2 <МГУ>

Обработка ИЛИ-запросов

- Аналогично обрабатываем запросы-дизъюнкции
- Запрос: *московский* && (*институт* || *университет*)
- Применяем логические операции:
  - *институт* → 100
  - *университет* → 011
  - <ИЛИ> = 111
  - Московский → 110
  - <И> = 110

||

&&

- 110 соответствует 1-му и 2-му столбцам: {№1<МФТИ>, №2 <МГУ>}

Булева модель: достоинства и недостатки

- Достоинства:
  - Простота
  - Пользователь получает полный контроль над результатами поиска
  - Идеальная область применения: пользователи-эксперты, которые работают с небольшой коллекцией документов
- Недостатки:
  - “Обычные” пользователи не любят писать сложные логические выражения
  - Булева модель плохо работает для больших коллекций

Булева модель: избыточная полнота и точность

- Булевы запросы являются слишком “мягкими” и находят большое число малорелевантных документов в больших коллекциях
  - Пример: запрос *мгу* в веб-поиске (миллионы документов в коллекции, но пользователей интересуют только несколько самых релевантных)
- Верно и обратное: булевы запросы являются слишком “жесткими”
  - По запросу *лучшие университеты россии* мы не найдем документ *московский государственный университет*, если в нем нет слова *россия*
  - На практике пользователю очень сложно подобрать “руками” нужный баланс между точностью и полнотой

Современная поисковая система

- Что нужно пользователю?
- Запросы в свободной форме
  - Без логических операторов
  - Смысл запроса!
- Ранжирование
  - Топ-10 из миллионов с ключевыми словами
- Скорость!
  - Время ответа <1с
- Под капотом любого современного поисковика работает булев поиск

Булев поиск сегодня

- “Мягкая” булева модель
  - На практике, в современных поисковиках работает «мягкий» вариант булевой модели
  - Оператор «мягкого» И – требуем, чтобы документ содержал не все, а только самые важные (про важность – в следующих лекциях) из слов запроса

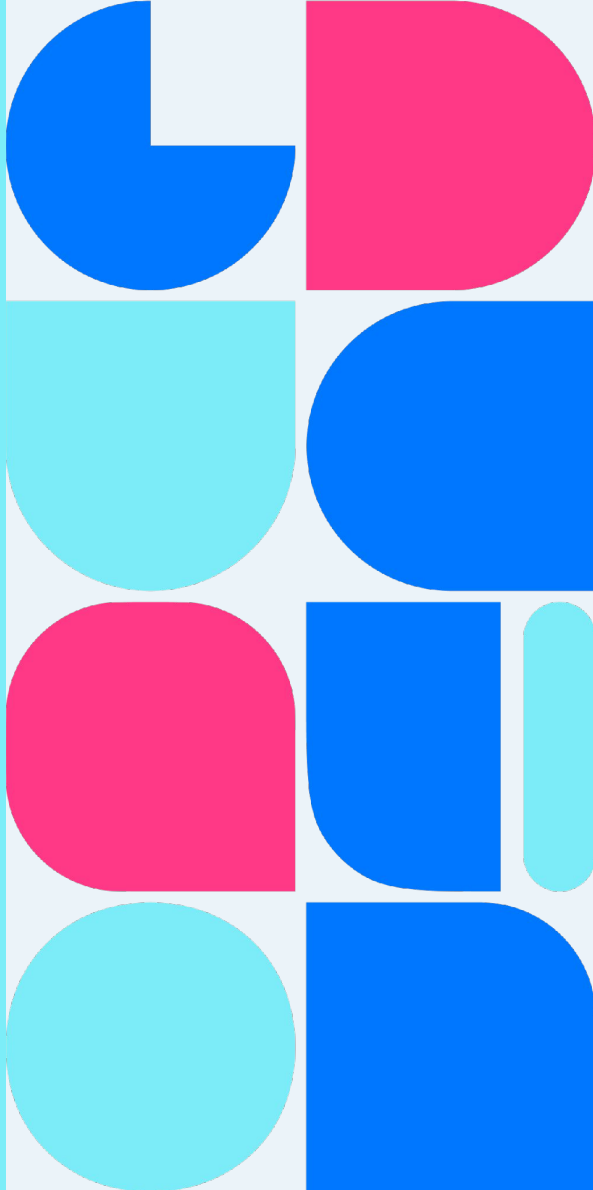
Повторение из лекции 1

Итоги раздела: булев поиск

- ✓ Разобрали, как документы и запросы превращаются в термины
- ✓ Изучили и-, или- и сложные запросы и дерево запроса
- ✓ Научились искать документы по матрице «термин — документ»
- ✓ Поняли, где булев поиск ломается: избыточная полнота и недостаточная точность
- ✓ Выяснили, что современные поисковики используют мягкую версию булева поиска

Обратный индекс

Разобраться, как устроен обратный индекс и почему без него поиск не масштабируется

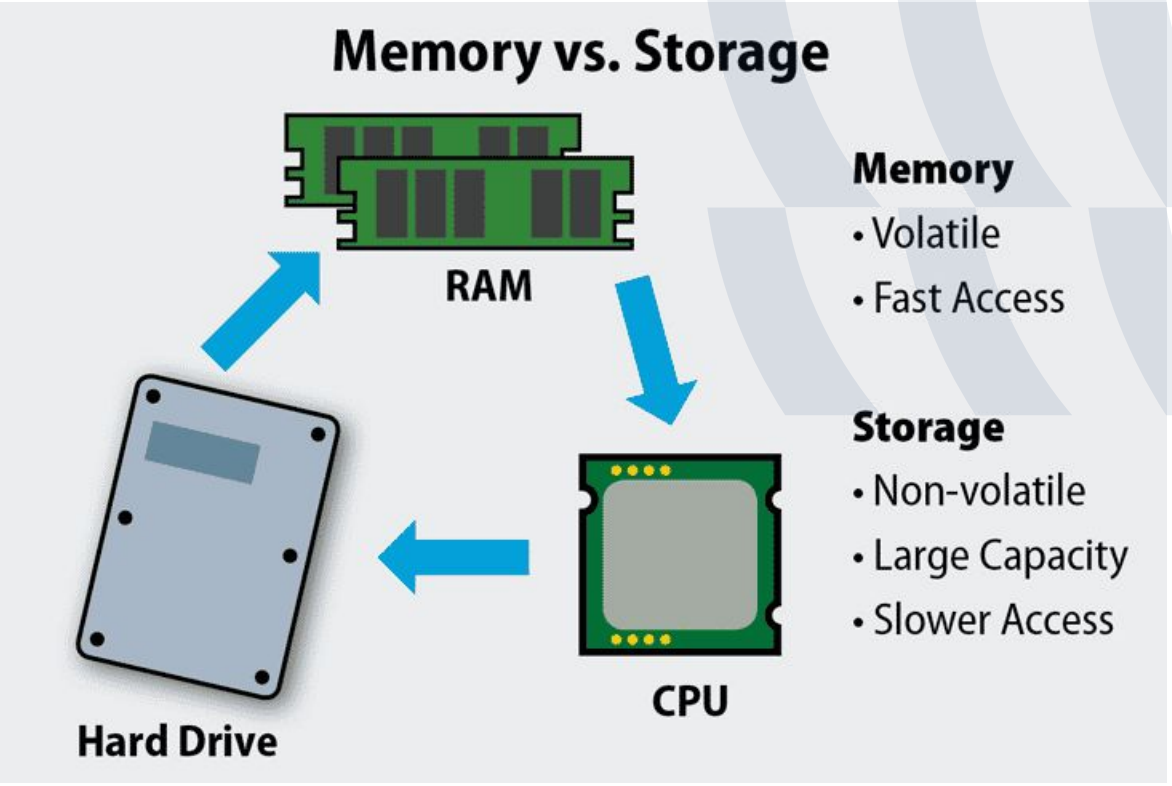


Как реализовать булеву модель?

- Существует эффективная реализация булевой модели на основе обратного индекса
- Но сначала:
  - Вспомним как работает современное компьютерное железо
  - Рассмотрим несколько наивных и неэффективных решений

Современный компьютер

- Нам будут интересны только:
  - Процессор (CPU)
  - Оперативная память (RAM)
  - Внешняя память (HDD, SSD)



Память: RAM, SSD и HDD

RAM

доступ ≈ ~100 нс

- \+ Очень быстрый произвольный доступ
- \+ Очень надёжна
- – Мало: обычно < 512 ГБ на сервер
- – Данные не переживают перезагрузку
- – Дорогая

SSD

доступ ≈ ~100 мкс

- \+ Средний объём, обычно ~1 ТБ
- \+ Случайный доступ в 100 раз быстрее HDD, у NVMe — до 10 мкс
- \+ Часто ставят кешем перед HDD
- – Всё ещё сильно медленнее RAM
- – Ограниченный ресурс записи
- – Дороже HDD

HDD

доступ ≈ до 10 мс

- \+ Большой объём: обычно > 8 ТБ
- \+ Дёшево, легко собирается в RAID
- \+ Данные переживают перезагрузку
- – Медленное последовательное чтение
- – Крайне медленный произвольный доступ
- – Ломаются часто

Иерархия памяти

- Типичные времена ожидания для разных типов оперативной и долговременной памяти

Память	Время ожидания	Комментарий
L1 cache	0.5ns	
L2 cache	7ns	
RAM	100ns	200x L1 cache
Compress 1 KB (snappy)	3 µs	Fast!
Network (send 1 KB over 1Gbps)	10 µs	
SSD (read 4 KB randomly)	150 µs	~1 GB/s
HDD seek	10 ms	
HDD (read 1 MB sequentially)	20 ms	

Память — самый ценный ресурс!

- Памяти всегда мало
- Единственная возможная стратегия:
  - Большие «холодные» данные лежат на HDD
  - Маленькие «горячие» данные грузим в RAM
  - Используем SSD для «теплых» данных или под кэш

Наивный подход к обработке булевых запросов

- Сколько памяти надо для хранения матрицы термин-документ?
- Типичная маленькая коллекция
  - 1 млн документов, 1 млрд терминов (~1 млн уникальных)
  - #(Элементов матрицы Т-Д) = 1 млн док-тов * 1 млн уник терминов = 1 трлн
  - Потребуется 128 Гб из расчета 1 бит/элемент – это 1 мощный сервер
- Не будет работать в масштабах веб-поиска
  - 10 млрд док-тов в Рунете: > 1 PB RAM!!!

Почему просто не использовать grep?

- Храним документы как текстовые файлы на диске
- Запрос: *московский университет*
- Делаем *grep университет *.txt | grep московский*
- Типичная маленькая коллекция
  - 1 млн документов * 1 млрд терминов * 10 байт/термин = 10 Гб на диске
  - Типичная скорость последовательного чтения с HDD - ~150 Мб/с
  - grep с HDD займет > 1 минуты
  - Не будет работать даже для маленькой базы!

Повторение из лекции 1

Рабочее решение: обратный индекс

- Основная идея: матрица термин - документ ОЧЕНЬ разреженная
- Большинство слов встречаются только в очень небольшом числе документов
- Будем хранить только ненулевые элементы!
- Отображение: термин → список из ID документов, содержащих этот термин

Повторение из лекции 1

Обратный индекс: пример

- *государственный* → [2]
- *институт* → [1]
- *итмо* → [3]
- *московский* → [1, 2]
- *технический* → [1]
- *физика* → [1]
- *университет* → [2, 3]

Документы:
1. <МФТИ>
2. <МГУ>
3. <ИТМО>

- **Словопозиция** (posting) – элемент списка с информацией о наличии термина в док-те (в нашем примере просто DocID)
- **Словарь** – структура данных для хранения терминов

Повторение из лекции 1

Сколько нужно памяти под обратный индекс

- Точное потребление памяти зависит от распределения терминов
- Типичный случай: размер индекса =~10% от размера коллекции
- Большая коллекция веб-масштаба
  - 10 млрд док-тов * 1000 терминов/док-т * 10 байт/термин = 100 Тб
  - Размер индекса = 0.1 * 100 Тб = 10 Тб
  - Немало, но гораздо лучше чем было в наивном варианте

Как хранить мультитерабайтный индекс?

- Предположим, что мы хотим полностью загрузить индекс в память
  - У нас нет машины с 10 Тб памяти
  - Но мы можем «порезать» (шардировать) индекс по кластеру
  - 10 Тб / 48 Гб/сервер =~ 200 серверов
  - На практике еще меньше т.к. существуют эффективные алгоритмы сжатия

Как построить обратный индекс

- Начнем с простейшего случая
- Индекс полностью помещается в память на 1 машине
- Построение индекса также полностью происходит в памяти

Построение обратного индекса на Python

- Используем словарь из множеств (dict of sets) для сбора информации по каждому термину

```
def build_index(docs):
    inverted_index = {}
    next_doc_id = 1
    for doc in docs:
        for word in doc:
            postings = inverted_index.setdefault(word, set())
            postings.add(next_doc_id)
        next_doc_id += 1
    return inverted_index

inverted_index = build_index([
    ["moscow", "institute", "of", "physics", "and", "technology"],
    ["moscow", "state", "university"],
    ["itmo", "university"]
])
print(inverted_index)
```

Обработка запроса с помощью обратного индекса на Python

- Ищем в словаре множества словопозиций для каждого из терминов запроса
- Пересекаем множества с помощью оператора &

```
def search(query, inverted_index):
    results = None
    for word in query:
        postings = inverted_index.get(word, set())
        results = results & postings if results is not None else postings
    return results

results = search(["moscow", "university"], inverted_index)
print(results)
```

Итоги раздела: обратный индекс

- ✓ Вспомнили, как устроена память: RAM, SSD, HDD и их времена доступа
- ✓ На цифрах увидели, почему матрица «термин — документ» и grep не масштабируются
- ✓ Разобрали обратный индекс: словарь терминов → списки словопозиций
- ✓ Оценили размер индекса — примерно 10 % от размера коллекции
- ✓ Построили простой индекс и поиск по нему на Python

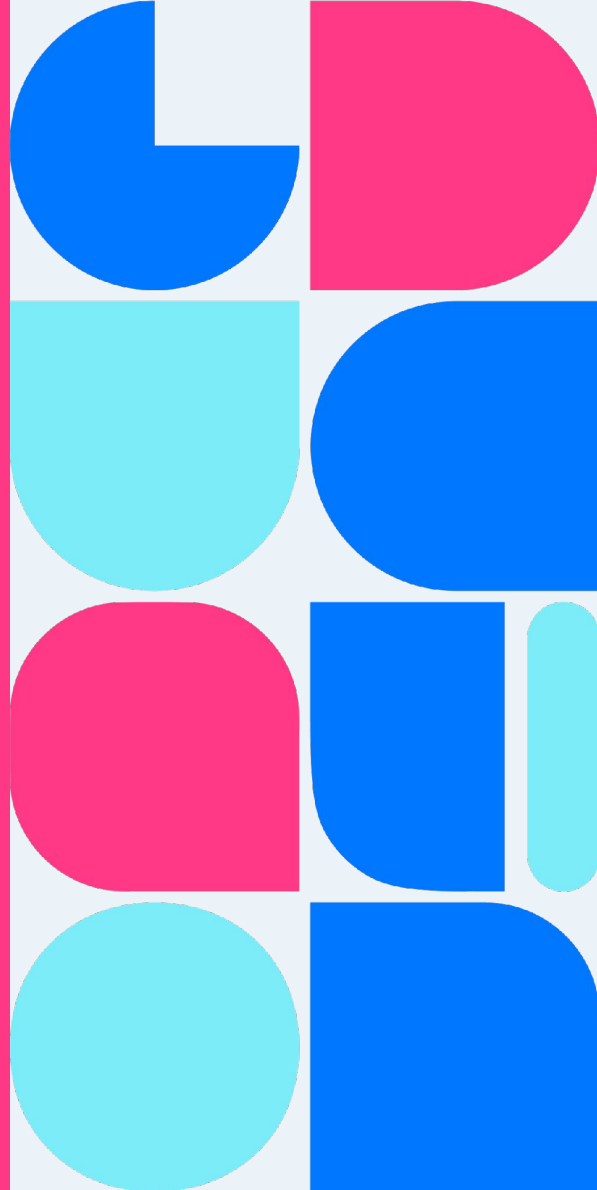
Вопросы?



education

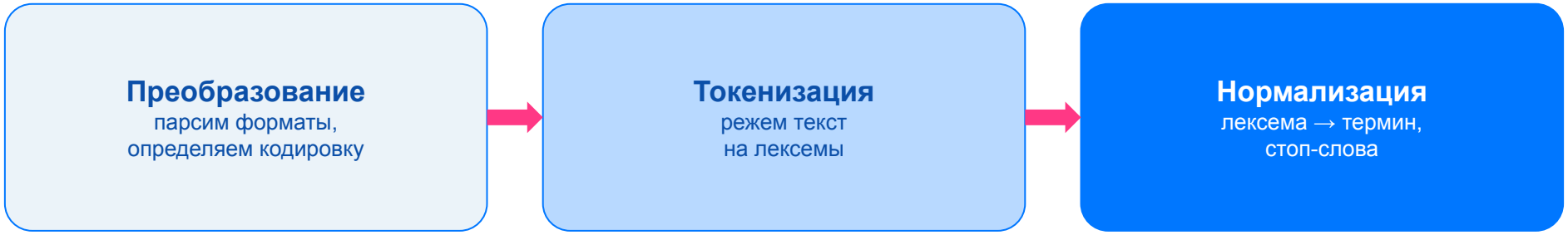
Препроцессинг документов

Разобраться, как из сырого документа получить набор терминов для поискового индекса



Предобработка документов

- Мы рассматривали запросы и документы как последовательности терминов
- Но остались открытыми вопросы: как превратить сырой документ в последовательность терминов и как именно нормализовать слова
- Ответ — конвейер из трёх этапов:



На входе — сырой документ (HTML, PDF, DOC…), на выходе — список терминов для индекса

Этап №1: преобразование документа

- Поисковый робот качает документы в разных форматах
  - Текстовые (HTML) или бинарные (DOC, PDF, ...)
  - Кодировка тоже бывает разной: UTF-8, CP1251, ...
- На входе “сырые” бинарные документы
- Парсим разные форматы и достаем текст
- Определяем исходную кодировку текста и конвертируем его во внутреннюю кодировку поисковой системы (напр. UTF-8)
- На выходе: текст во внутренней кодировке

Этап №2: токенизация

- Разбиваем текст на **лексемы (токены)**
- **Лексема** – это экземпляр последовательности символов, объединенных в семантическую единицу для обработки (Manning)
- Лексемы, как правило, соответствуют словам
- На входе: Московский физико-технический институт (МФТИ)
- Бьем по пробелам, удаляем знаки пунктуации
- На выходе список лексем: [Московский, физико,технический, институт, МФТИ]
- Много пограничных случаев: как быть с C++?
- Зависит от языка → нужен детектор языка!
- Например, в китайской письменности нет пробелов – токенизация может быть разной в зависимости от контекста

Этап №2: токенизация — разные языки

- Китайский язык необычен тем, что у каждого символа есть смысл, но это не значит, что каждый символ (иероглиф) может быть воспринят как слово.
- Кроме того, значение слова не обязательно связано со значением иероглифов, составляющих его.
- Например, китайское слово кризис 危機 состоит из иероглифа “опасность” и иероглифа “возможность”. Однако второй иероглиф имеет значение ближе к “важный момент”, и значит “машина” в других контекстах.
- Этот же символ присутствует как первый иероглиф в слове “аэропорт” 機場, в котором второй иероглиф значит “поле”.
- Соединение частей слова правильным образом – очень важно, т.к. аэропорт это не “машинное поле”. Свобода 自由 - это не синоним для “само причина”.
- Многие слова в китайском состоят из 2 иероглифов, но не все. Поэтому для китайского хорошо работает биграмный индекс

Этап №2: токенизация - пограничные случаи

- Все ли лексемы мы хотим добавлять в обратный индекс?
- E-mails
- URLs
- IP адреса
- Трекинг номера
- Финансовые суммы
- Числа
- Сильно раздувают размер словаря
- Можно исключать, но теряем возможность искать по ним

Этап №3: нормализация

- Приведение лексем к нормальной форме
- На входе список лексем: [*Московский*, *физико*, *технический*, *институт*, *МФТИ*]
- Приводим к нижнему регистру
- Нормализуем: лексема → нормальная форма (стемминг или лемматизация, см. дальше)
- Удаляем стоп-слова (самые частотные, однобуквенные, …)
- На выходе список терминов: [*московский*, *физика*, *технический*, *институт*, *мфти*]

Нормализация с помощью стемминга

- Стемминг – приближенный эмпирический процесс, в ходе которого от слов отбрасываются окончания в расчете на то, что это себя оправдает (Manning)
- Например:
  - fishing → fish
  - fished → fish
  - fisher → fish

Стемминг: плюсы и минусы

+ Прост в реализации
+ Работает быстро
+ Хорошие результаты для “простых” языков (английский)

- Много коллизий у разных по смыслу слов (напр. стеммер Портера приводит universe и university к univers)
- Плохо работает для «сложных» языков, в которых у слова может быть множество форм (русский, финский, ...)

Лемматизация

- Лемматизация – это точный процесс с использованием лексикона и морфологического анализа слов, в результате которого удаляются только флективные окончания и возвращается основная форма слова, называемая леммой (Manning)
- Например:
  - рыбаки → рыбак
  - рыбаками→ рыбак
  - рыбаков → рыбак
- Плюс: хорошо работает для русского и других “сложных” языков
- Минусы:
  - Сложна в реализации, требует морфологического анализа слова
  - Мало качественных библиотек
  - Работает медленно
  - Для “простых” языков показывает результаты не сильно лучше стемминга
  - Бывают ошибки: Киев → Кий, Меган → Мигать

Стемминг или лемматизация?

- Для английского языка лучше начинать со стемминга, и только при необходимости переходить к лемматизации
- Для русского языка необходимо использовать лемматизацию
- Все большие поисковые машины используют лемматизацию («морфология»)

Предобработка запросов

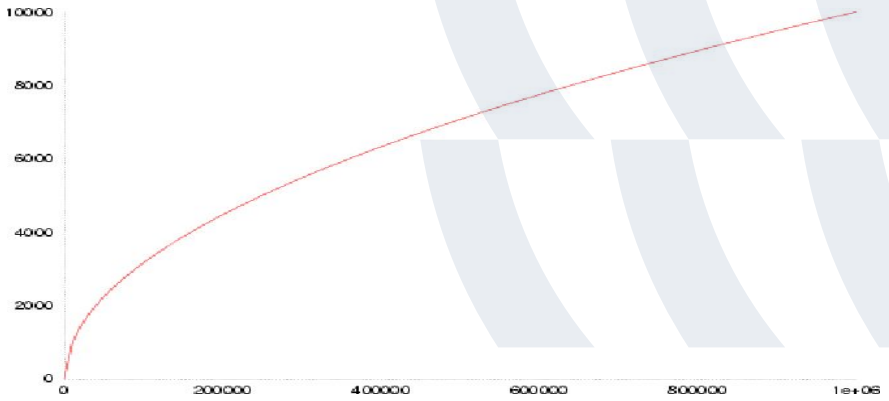
- Симметричной предобработке подвергаются и запросы
- Очень важно, чтобы токенизация и нормализация для запросов и документов были идентичны

Закон Хипса (Heaps' Law)

- Как растёт словарь по мере того, как мы индексируем всё больше текста

$$|V| = k \cdot T^{b}$$

$30 \le k \le 100$, $b \approx 0.5$ · $|V|$ — размер словаря, T — число лексем в коллекции



Что из этого следует

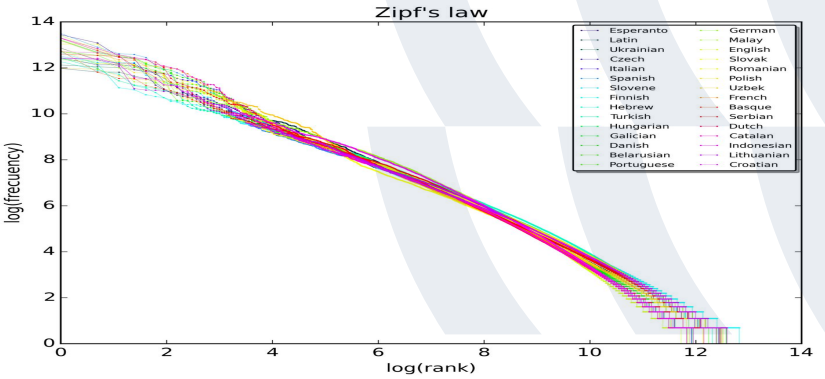
1. Словарь не перестаёт расти никогда: новые термины появляются на любом объёме
2. Но растёт медленнее коллекции — примерно как корень: текста в 4 раза больше → словарь вдвое
3. Значит, размер словаря нельзя зафиксировать заранее — структура должна быть расширяемой

Закон Ципфа (Zipf’ Law)

- Как частоты терминов распределены внутри коллекции

$$cf_i \propto 1 / i$$

cf_i — частота i-го по популярности термина · степенной закон с k ≈ 1



Что из этого следует

1. Частоты падают обвально: второй по популярности термин встречается вдвое реже первого
2. Горстка терминов покрывает огромную долю всех вхождений — отсюда стоп-слова
3. А у длинного хвоста редких терминов списки короткие — это то, что делает индекс сжимаемым

Итоги раздела: препроцессинг документов

- ✓ Прошли конвейер: преобразование → токенизация → нормализация
- ✓ Разобрали пограничные случаи токенизации и особенности разных языков
- ✓ Сравнили стемминг и лемматизацию: для русского языка нужна лемматизация
- ✓ Запомнили правило: препроцессинг запросов и документов должен быть идентичным
- ✓ Познакомились с законами Хипса и Ципфа

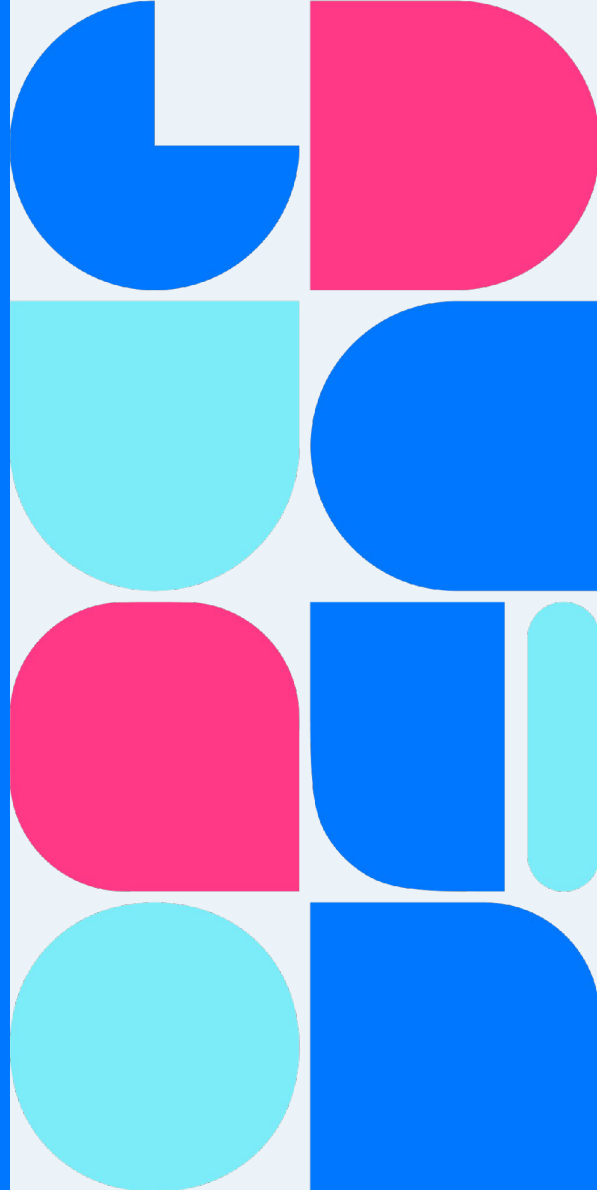
Вопросы?



education

Структура обратного индекса

Понять, из каких структур данных собран индекс и почему от них напрямую зависит скорость поиска

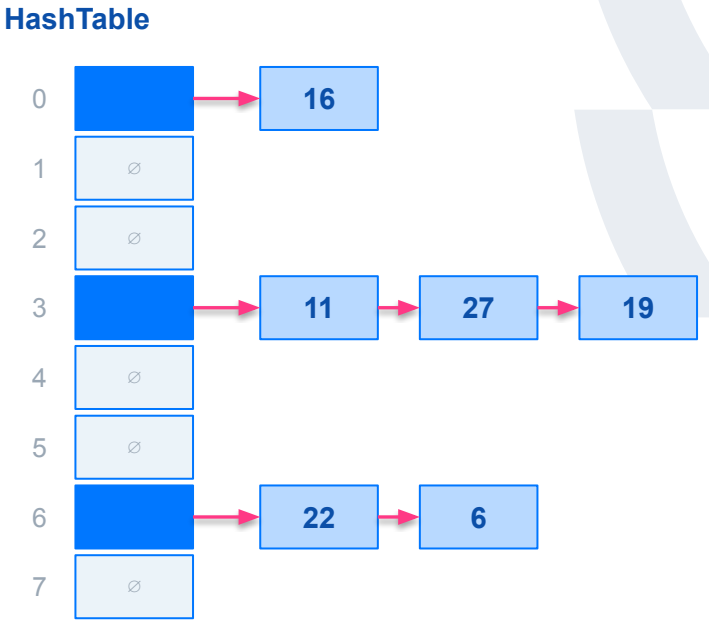


Какую структуру данных выбрать для словаря терминов?

- У нас миллион уникальных терминов, и по приходящему слову надо быстро найти, где лежит его список словопозиций. Вот этот словарь терминов и надо во что-то уложить.
- Возможны несколько вариантов хранения множества терминов:
  - Хэш-таблица
  - Дерево поиска (сбалансированное дерево поиска, B-дерево, ...)
  - Отсортированный список или массив
- Важный момент: словарь почти всегда помещается в память
  - Даже для веб-поиска размер словаря $|V| < 100$ млн терминов

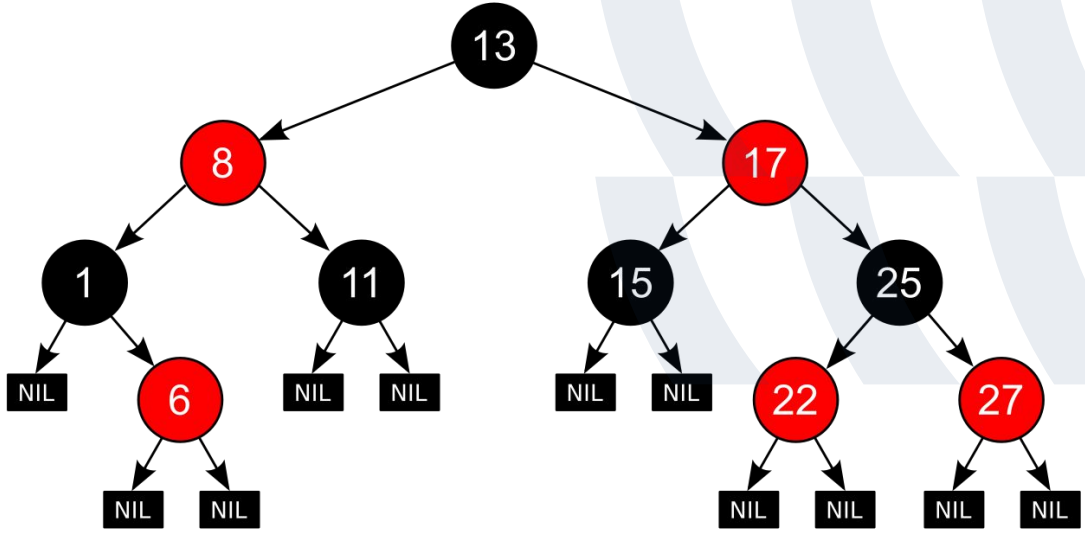
Словарь на основе хэш-таблицы

- dict в Python использует хэш-таблицу «под капотом»
- Быстрые вставка и поиск за O(1)
- Память используется не оптимально
- Хорошее решение для небольших индексов



Словарь на основе дерева поиска

- Вставка и поиск за O(log(N))
- Упорядочены: возможен поиск по префиксу!
- Много памяти расходуется на указатели
- «Рыхлые»: недружелюбны к кэшу ЦПУ
- Плохое решение если не нужен порядок

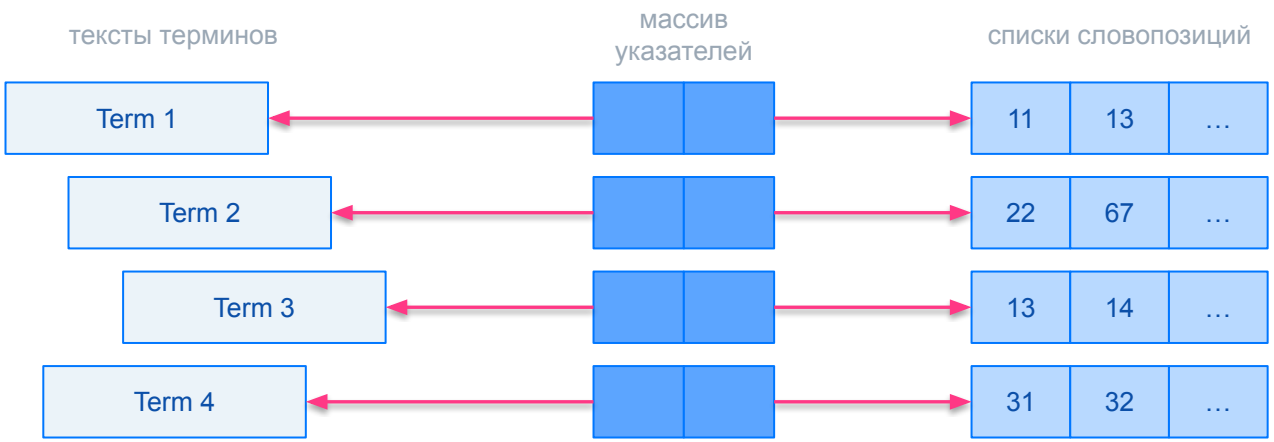


Словарь на основе отсортированного списка

- Динамической список с амортизированной стоимостью вставки O(1)
- Сортируем 1 раз, в конце процесса индексации перед началом обработки запросов
- Ищем термины бинарным поиском за O(log(N))
- Главная проблема: все термины разной длины!
- Мы не можем хранить такой список в непрерывном блоке памяти (массиве в языке C)

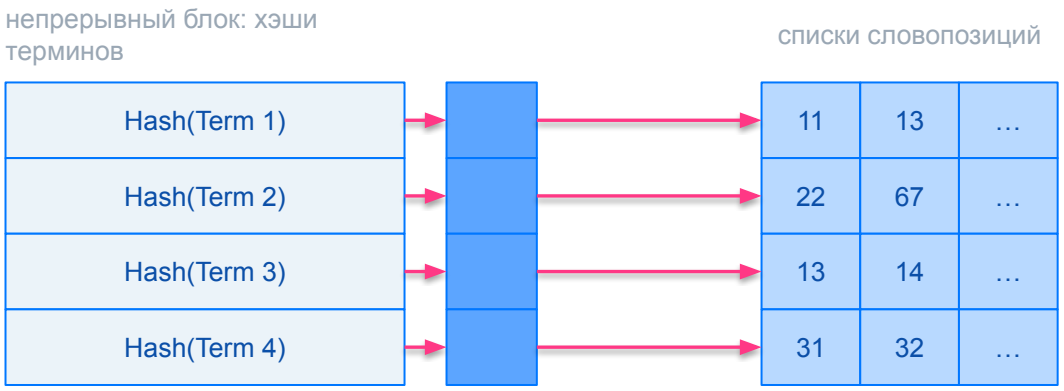
Динамический список в памяти (на примере list из Python)

- Нам понадобится динамический список наподобие list из Python
- Массив из указателей на текст термина и список словопозиций
- «Рыхлый», неоптимальный по кэшу и памяти



Динамический массив в памяти

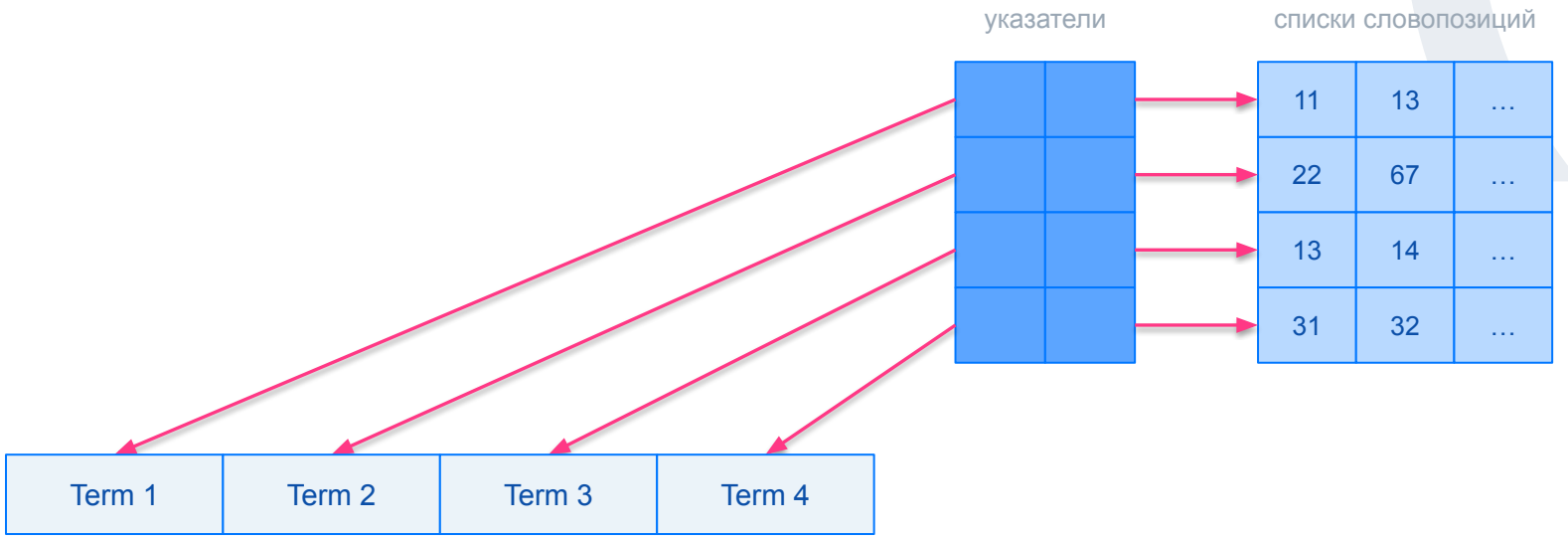
- Основная идея: надо хранить термины в непрерывном блоке памяти
- Не list из Python, а массив из C/C++!
- Можем пренебречь коллизиями и хранить хэши терминов



- Возможно ли обойтись без коллизий? (ответ на след. слайде)

Словарь как строка

- Можно хранить тексты терминов подряд в одном длинном блоке памяти
- В другом блоке храним указатели на начало терминов и списки словопозиций
- Отличное решение для больших индексов



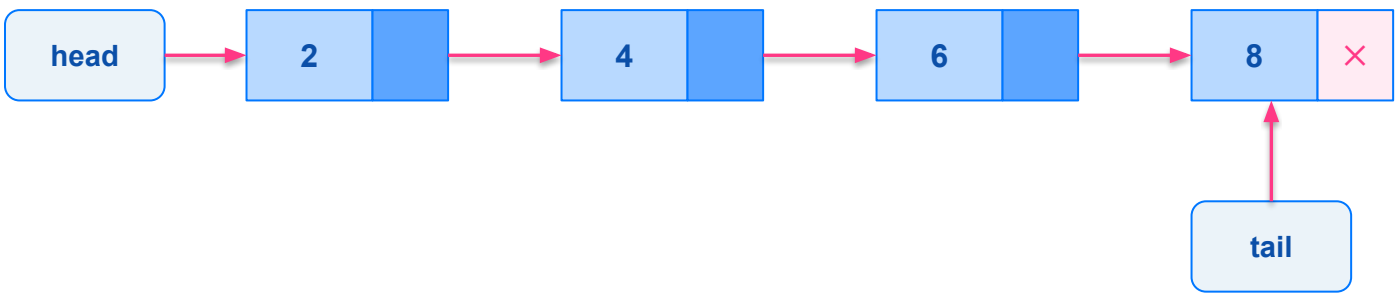
один непрерывный блок памяти с текстами терминов

Какую структуру данных выбрать для списка словопозиций?

- Возможны несколько вариантов
  - Связанный список
  - Хэш-таблица
  - Отсортированный массив
- Выбранная структура данных должна обеспечивать эффективное пересечение/слияние списков для многословных запросов

Списки словопозиций на основе связанных списков

- Простейшая структура данных
- Теряем много памяти под указатели
- Плохо используем кэш ЦПУ
- Почти всегда плохое решение на современном железе



на каждый элемент — лишний указатель, узлы разбросаны по памяти

Списки словопозиций на основе хэш таблиц

- Решение из нашего примера на Python
- Быстрые вставка и поиск за O(1)
- Быстрое пересечение за O(N): бежим по первой хэш-таблице и ищем элементы во второй
- Не совсем оптимально по памяти
- Отличное решение для небольших коллекций
- Не самое лучшее решение для больших!

Списки словопозиций на основе отсортированного массива

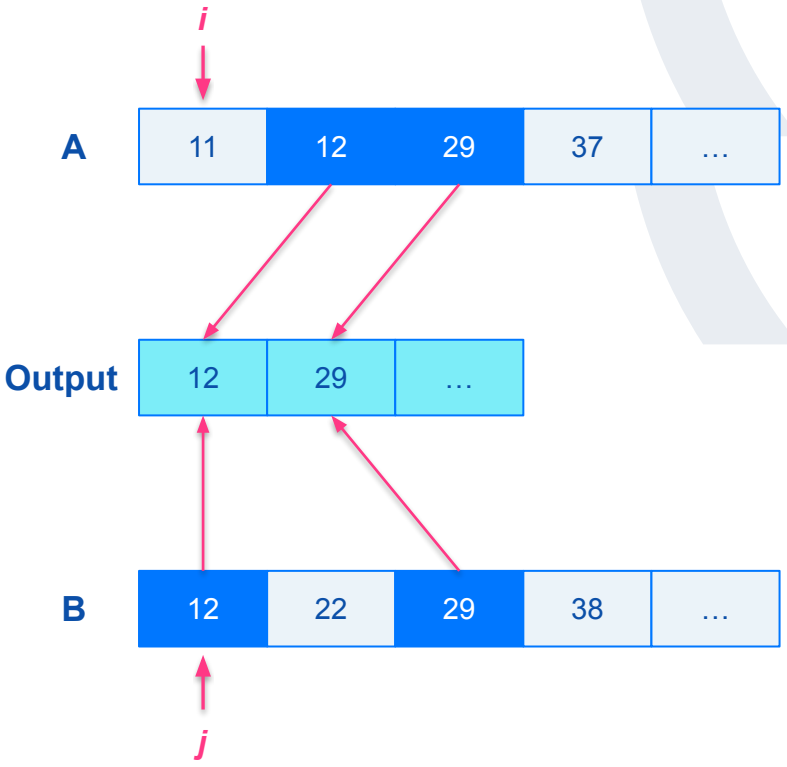
- Динамический массив из DocID: list в Python или std::vector в C++
- Быстрая вставка в конец с амортизационной стоимостью O(1)
- Сортируем за O(N*log(N))
- Сортированные массивы можно эффективно пересекать за O(N) времени и O(1) памяти
  - Подробнее позже
- Сортировку можно делать на этапе построения индекса!
- Лучшее решение для больших коллекций

Пересечение списков словопозиций

- Для обработки булевых запросов надо пересекать или сливать связанные с терминами списки словопозиций
- Простейший случай: 2-словные запросы
  - И-запросы: пересечение 2-х сортированных списков
  - ИЛИ-запросы: слияние 2-х сортированных списков
- Эффективный алгоритм: O(N1 + N2) по времени и O(1) по памяти

Пересечение 2-х сортированных списков

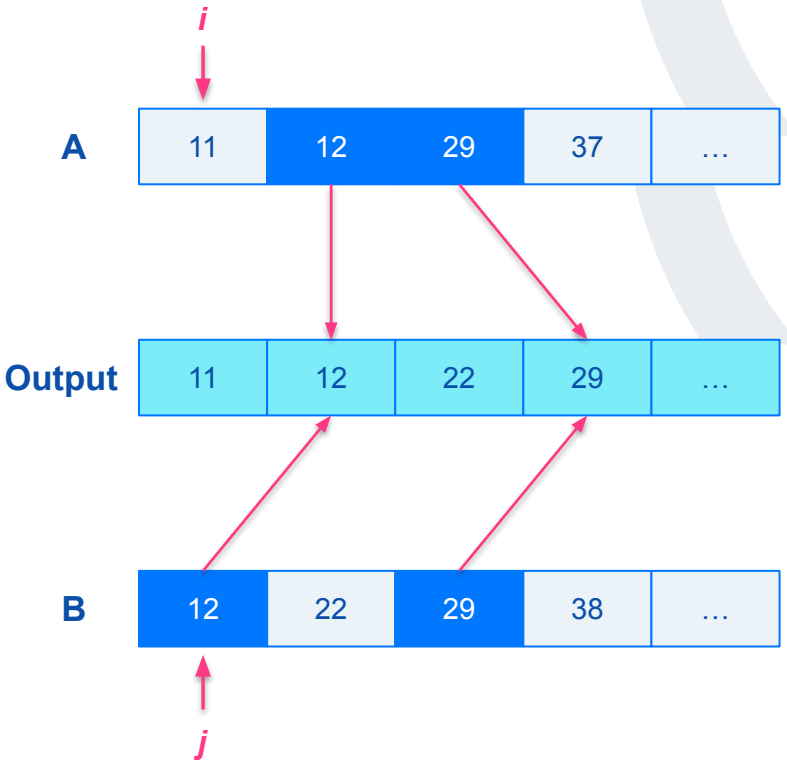
- И-запросы (конъюнкция)
- 2 индекса: i и j
- A[i] < B[j] ⟶ i++
- A[i] > B[j] ⟶ j++
- A[i] == B[j] ⟶ result.append(A[i]), затем i++ и j++



в результат попадают только общие docID

Слияние 2-х сортированных списков

- ИЛИ-запросы (дизъюнкция)
- 2 индекса: i и j
- A[i] < B[j] → result.append(A[i]), i++
- A[i] > B[j] → result.append(B[j]), j++
- A[i] == B[j] → result.append(A[i]), затем i++ и j++



12 и 29 есть в обоих списках — в результат они попадают один раз

Пересечение списков: почему «в лоб» дорого

Запрос «игра керлинг»: у «игры» ~10^6 документов, у «керлинга» — единицы

- Обычное пересечение двумя указателями — O(|A| + |B|): пройдём весь длинный список ради двух чисел
- Идея: искать каждый элемент короткого списка в длинном бинарным поиском — O(|B| · log|A|)
- Но бинарный поиск здесь не работает по двум причинам — см. ниже



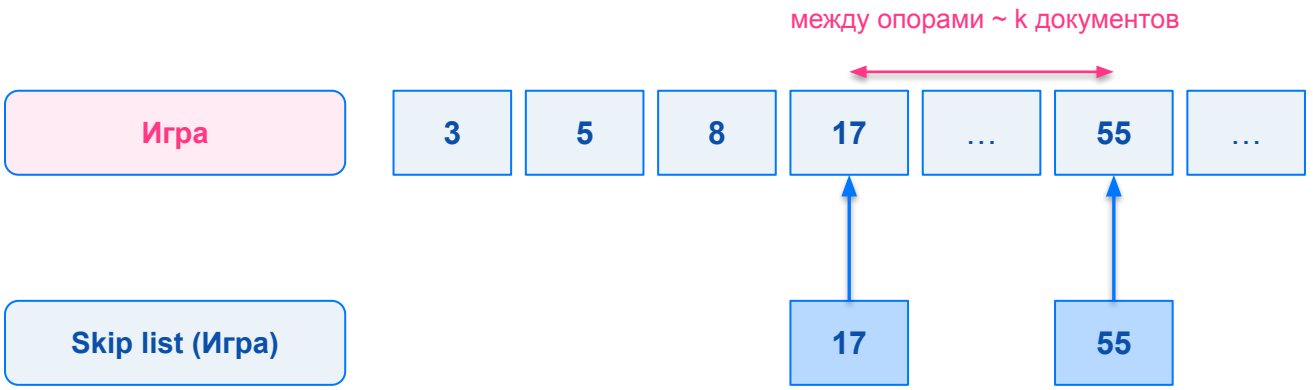
чтобы проверить два docID «керлинга», алгоритм двух указателей прочитает весь список «игры»

Почему бинарный поиск не спасает

1. Кэш: каждый шаг — прыжок в случайное место массива, промах кэша и непредсказуемое ветвление
2. Сжатие: список хранится дельтами переменной длины — к k-му элементу нельзя прыгнуть, не декодировав всё с начала

Skip-lists: прыгаем по опорам

- Рядом с длинным списком храним короткий массив опор: docID + смещение, по которому он лежит
- Опоры выбираются по позиции, а не по значению: **каждый k-й элемент списка**, классический выбор k = √L
- На практике опору ставят на границу сжатого блока — с неё декодер умеет стартовать
- Опоры считаются один раз при индексации

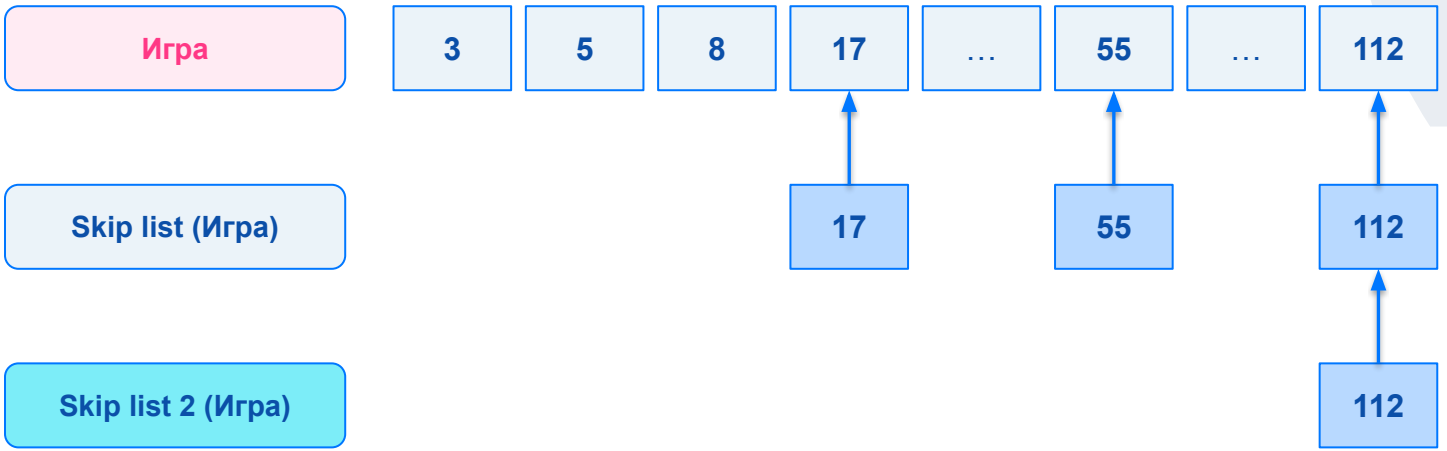


Ищем 55

сканируем опоры → последняя опора ≤ 55 → прыгаем по её смещению → дочитываем подряд не более k записей

Многоуровневый skip-list

- Один уровень сокращает 10^6 записей до ~10^3 опор — но и тысячу опор сканировать линейно уже долго
- Строим skip-лист над skip-листом: ~10^3 → ~30. При необходимости ещё уровень
- Каждый уровень — плоский массив, а не узлы с указателями: та же локальность, что мы требовали от словаря



Компромисс по шагу k

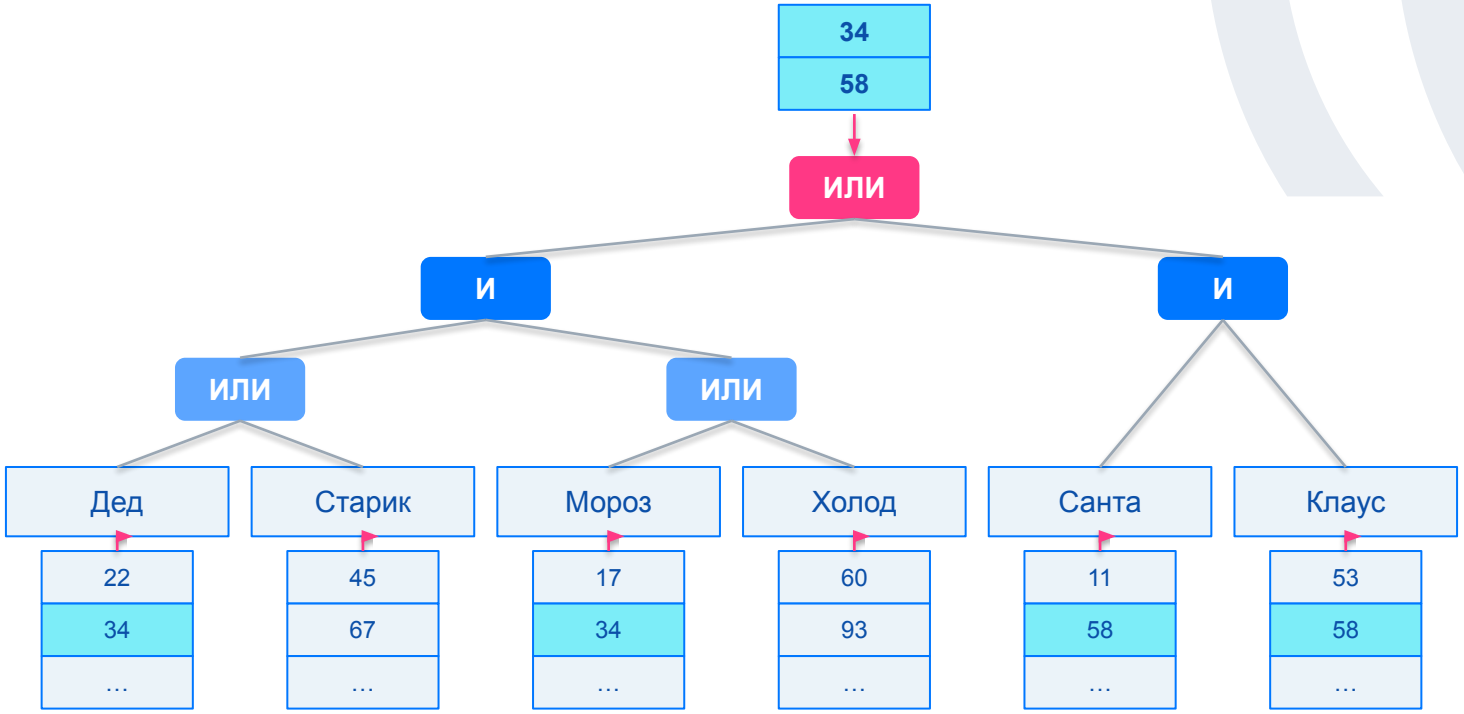
чаще опоры → быстрее прыжки, но больше памяти и хуже сжатие: каждая опора рвёт цепочку дельт

Стоп-слова

- Проблема: есть слова, которые встречаются почти в каждом документе
- Очень длинные списки словопозиций → очень медленное пересечение
- Возможное решение: списки стоп-слов
- Составляем список самых частотных слов: и, в, на, с, ...
- Правило 30: топ-30 самых частотных терминов порождают 30% лексем
- Просто удаляем такие слова из индекса
- Качество поиска практически не снижается

Обработка сложного дерева запроса

- С каждым термином-листом связан свой список документов
- Документы «текут» вверх по дереву: в каждом узле происходит пересечение (и) или слияние (или)



Фразовые запросы: два способа

Запрос «московский университет» — слова должны идти подряд. Но обычный обратный индекс хранит только факт «термин есть в документе»: порядок и соседство теряются. Нужен другой индекс.

Биграммный индекс

в словарь кладём пары соседних слов

- \+ «московский государственный» — фраза ищется как обычный термин, очень быстро
- – Словарь раздувается почти до $|V|^2$
- – Фразы длиннее двух слов — только приблизительно, бывают ложные срабатывания

Координатный индекс

храним позиции слов внутри документа

- \+ Храним docID + список позиций: фраза найдена, если позиции идут подряд
- \+ Работает для фраз любой длины и для «слова рядом»
- – Индекс заметно больше обычного

Запросы с ограничениями на близость слов

- Иногда требуется найти только те документы, в которых слова запроса находятся рядом
- «Рядом» можно определить строго: запросы удовлетворяет только те документы, в которых *distance(term_i, term_j)* <= K для любых i и j
- где distance(term_i, term_j) можно понимать как минимальное число слов в документе между терминами i и j

Расширенная булева модель

- Для обработки фразовых запросов и запросов с ограничениями на близость слов необходимо учитывать позиции слов в документе
- Такая модель иногда называется **расширенной булевой моделью**
- Будем использовать **координатный обратный индекс** – это обратный индекс, дополненный информацией о позициях (координатах) слов в документах

Координатный обратный индекс: пример

Словопозиции теперь содержат списки позиций термина в документе

- государственный → [2:[2]]
- институт → [1:[4]]
- итмо → [3:[2]]
- московский → [1:[1], 2:[1]]
- технический → [1:[3]]
- физика → [1:[2]]
- университет → [2:[3], 3:[1]]

Документы:
1. Московский физико-технический институт
2. Московский государственный университет
3. Университет ИТМО

Координатный обратный индекс гораздо больше обычного

- Позиция каждого слова из документа должна быть упомянута в списке позиций для соответствующего термина
- Размер индекса теперь пропорционален полному числу терминов в коллекции (а не числу уникальных терминов, как раньше)
- Размер такого индекса сравним в размером самой коллекции!
- И на порядок больше размера обычного индекса
- Памяти нужно еще больше
- VK хранит списки DocID в памяти (RAM), а списки позиций на SSD

Координатный обратный индекс необходим для ранжирования

- Модели поиска с ранжированием должны учитывать, что:
  - Документ, в котором слова запроса найдены в начале, как правило более релевантен запросу, чем документ со словами в середине или конце
  - Документ, в котором слова запроса идут подряд (или почти) и в таком же порядке (или почти) как правило более релевантен, чем документ, в котором эти же слова находятся далеко друг от друга

Обработка запросов с ограничениями на близость и порядок слов (в т.ч. фраз)

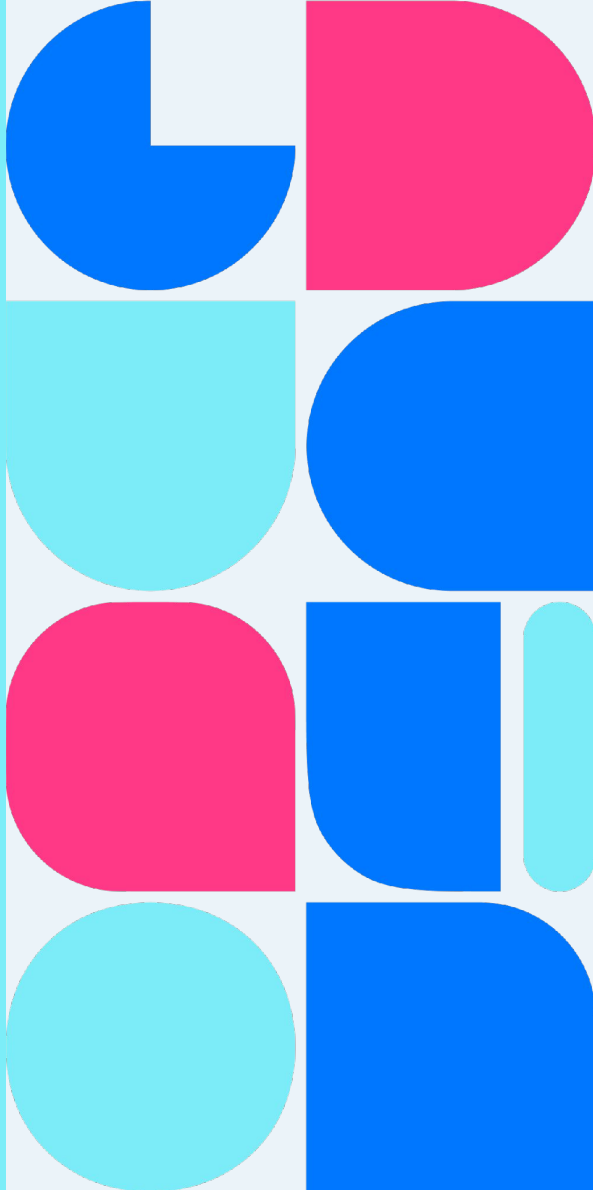
- Ищем и пересекаем/сливаем списки словопозиций так же, как и раньше
- Для каждого документа, который нашелся бы обычному булевому запросу:
  - Получаем списки позиций
  - Проверяем, насколько он удовлетворяет ограничениям на порядок и близость слов
  - Если да, то добавляем в список результатов поиска

Подведем промежуточные итоги

- Мы реализовали булеву модель поиска с помощью обратного индекса, и рассмотрели некоторые ее расширения
- Знаем, как строить обратный индекс, когда он еще помещается в память
- Все еще не знаем как строить большие обратные индексы, которые не помещаются в память

Построение больших индексов

Разобраться, как строить индекс, который не помещается в память одной машины



Построение больших индексов

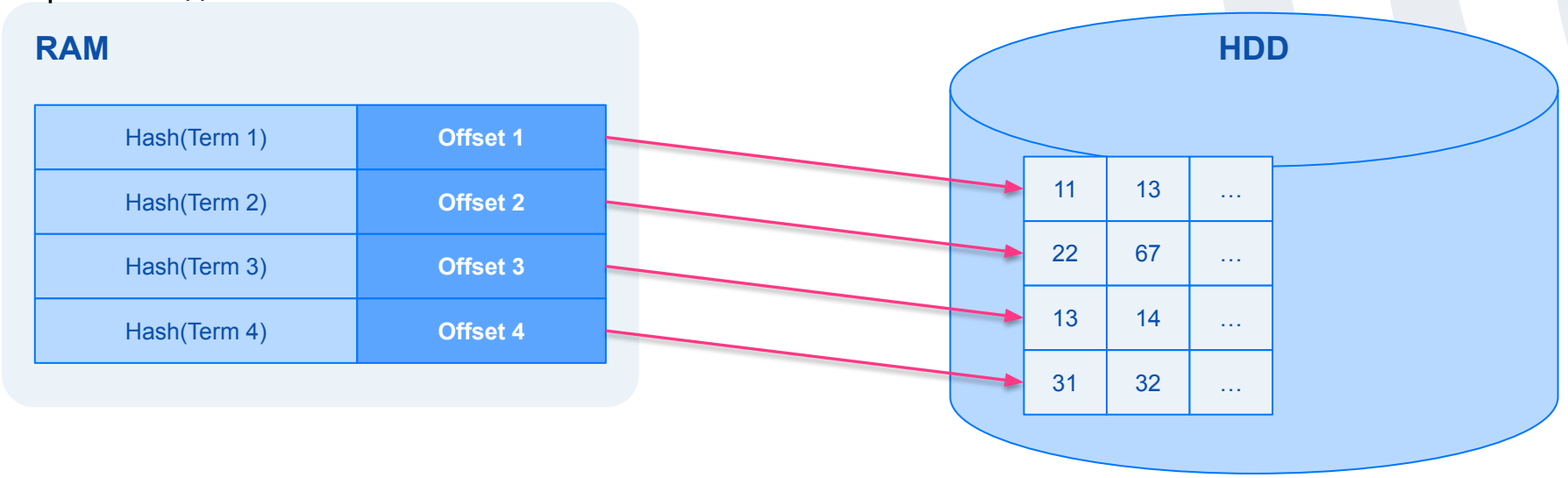
- Наши примеры обработки запроса и построения индекса на Python работают только в тех случаях, когда индекс помещается в память
- На практике это, как правило, не так
  - обратный индекс по Рунету занимает ~50 Тб
- Перейдем к рассмотрению алгоритмов построения таких индексов

Данные бывают большие и очень большие

- Будем различать 2 основных случая
  - Индекс не помещается в память, но все еще помещается на жесткий диск на одной машине
    - Индекс размером до нескольких терабайт
    - Обрабатываем запросы без загрузки индекса в память
    - Применяем алгоритмы индексации во внешней памяти
  - Очень большой индекс
    - Индекс размером десятки терабайт и больше
    - Обрабатываем запросы на поисковом кластере
    - Применяем распределенные алгоритмы индексации на кластере

Индекс со списком словопозиций на диске

- Словарь с оффсетами держим в памяти
- Сами списки словопозиций лежат в файле на диске

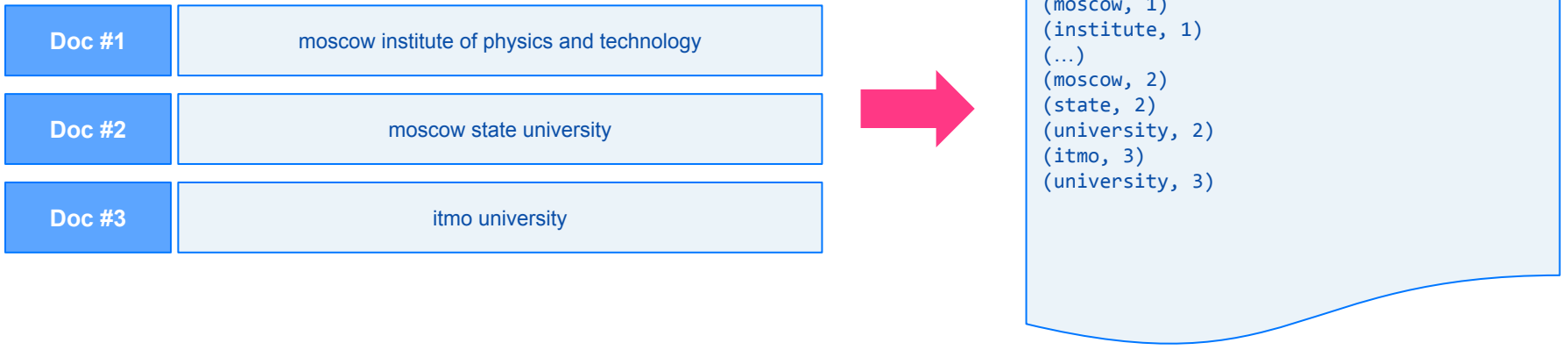


Индексирование, основанное на сортировке

- Индекс все еще помещается на жестком диске
- Решение основано на применении операций сортировки и слияния с использованием внешней памяти
- Состоит из 3-х этапов (детали на след. слайдах):
  - MAP: документ → список пар (термин, DocID)
  - SORT: сортируем пары на диске, в качестве ключа используем термин
  - REDUCE: группируем DocID’ы одного и того же термина в списки словопозиций

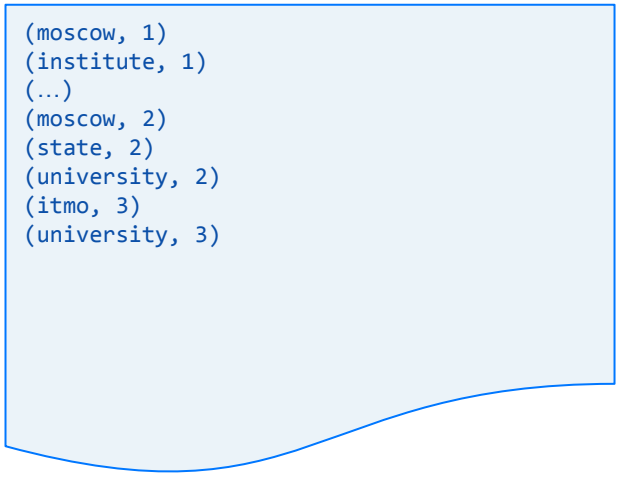
Этап #1: MAP

- Для каждого документа из коллекции: загружаем в память, токенизируем и нормализуем
- Для каждого термина порождаем пару (термин, DocID)
- Все пары дописываем на диск в один большой файл

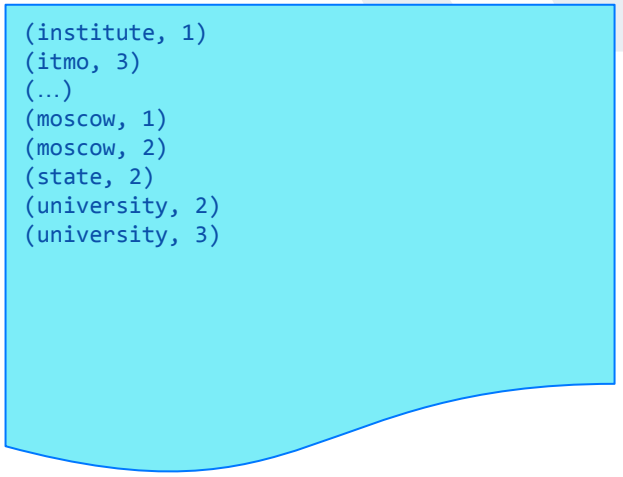


Этап #2: SORT

- Сортируем большой файл с парами на диске
  - Как, если размер файла больше доступной памяти?
  - Вернёмся к этому вопросу чуть позже



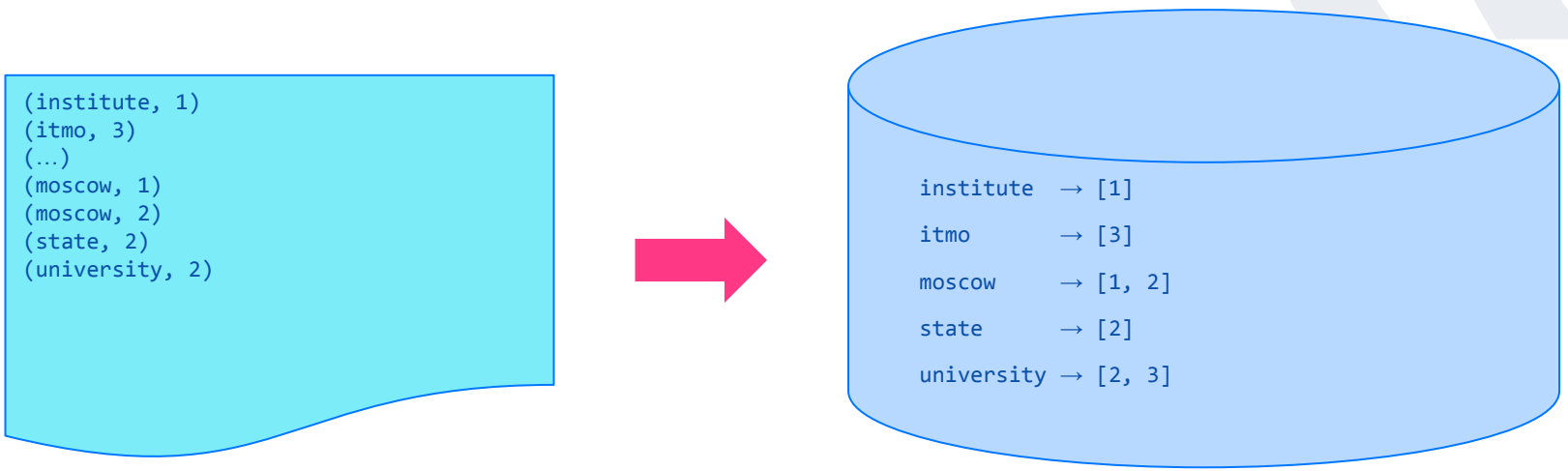
как есть



отсортировано по термину

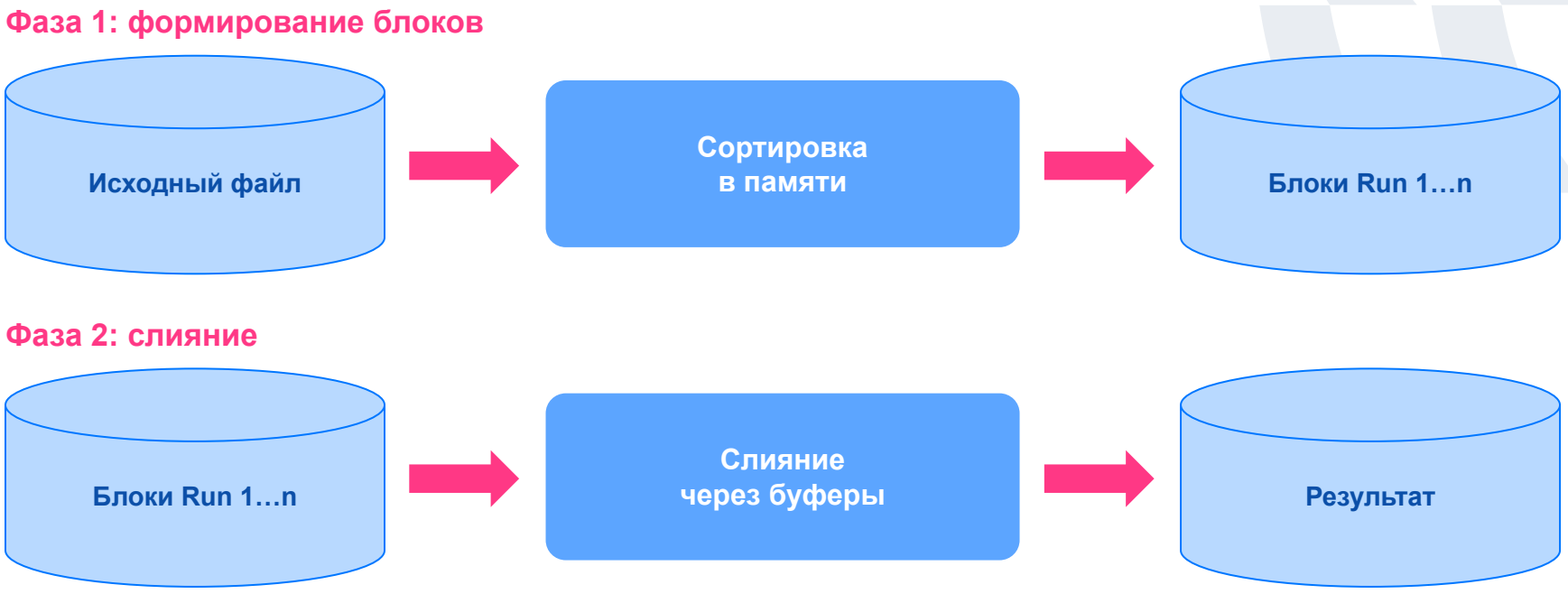
Этап #3: REDUCE

- Для каждого термина читаем из файла все связанные с ним пары
- Формируем список словопозиций
- Сохраняем термин и список словопозиций обратно на диск, в файл обратного индекса
- Эффективно по памяти: O(L), где L — длина списка словопозиций



Сортируем большой файл во внешней памяти

- Разбиваем файл на блоки размером с доступную память, сортируем каждый блок в памяти, затем сливаем отсортированные блоки



Утилита sort

- На практике для сортировки больших файлов во внешней памяти удобно использовать утилиту sort
- Файл $FILE с парами (term, count)
- Сортируем по термину: *sort –k1,1 $FILE*
- При желании, всю индексацию можно реализовать используя только утилиты UNIX: sort, uniq, awk

Обработка запросов на кластере

- Самый тяжелый случай: индекс не помещается на одной машине
- Решение: **шардирование**
- Разбиваем большой индекс на блоки-шарды
- Каждый шард храним на отдельной машине в кластере
- Возможны 2 способа разбиения: по терминам и по документам

“Нарезка” индекса по терминам

- В каждый шард попадает только часть обратного индекса для какого-то диапазона терминов
- Термины [А-К]
  - абажур → [771, ...]
  - аббат → [608, ...]
  - абзац → [133, ...]
  - ...
- Термины [Л-Я]
  - лабаз → [905, ...]
  - лабиринт → [108, ...]
  - лаборант → [297, ...]
  - ...

“Нарезка” индекса по документам

- Более сбалансированная схема
- В VK индекс нарезается так: shard = hash(DocURL) % (общее кол-во шардов)
- Современные поисковики используют подокументное шардирование

Шард №1

- абажур → [771, ...]
- аббат → [608, ...]
- абзац→ [133, ...]
- ...

Шард №2

- абажур → [1502, ...]
- аббат → [2913, ...]
- абзац → [96, ...]
- ...

Шард №3

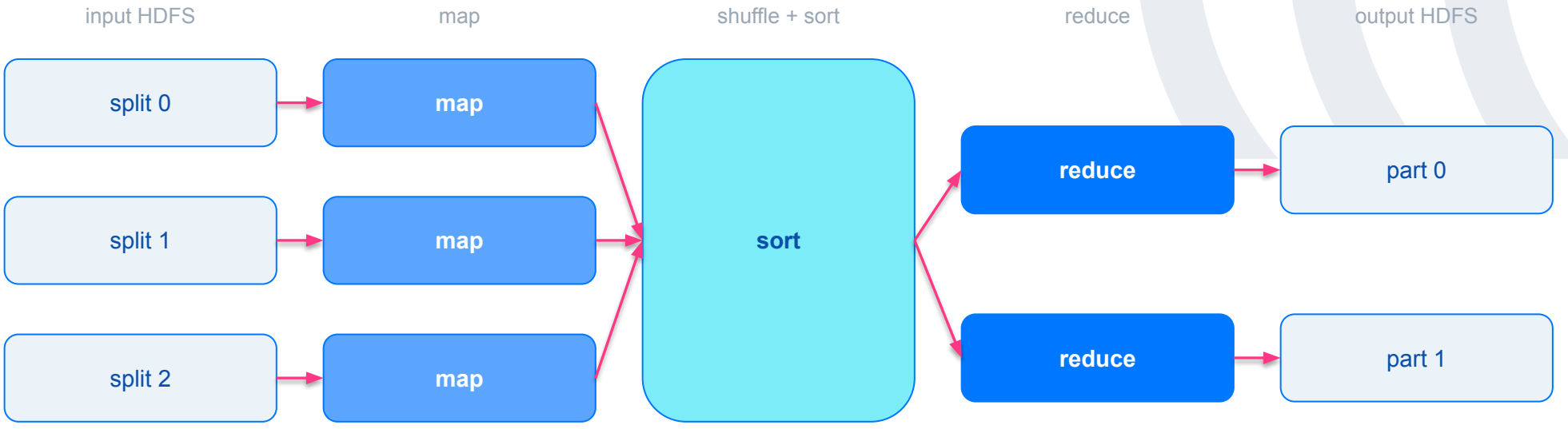
- абажур → [582, ...]
- аббат → [397, ...]
- абзац → [210, ...]
- ...

Распределенная индексация

- Самый тяжелый случай: индекс не помещается даже на диске
- Индексирование, основанное на сортировке – это один из вариантов парадигмы MapReduce
- Нам понадобится кластер с установленной реализацией MapReduce, напр. YT/Hadoop
- Алгоритм остается прежним, только:
  - Этапы MAP и REDUCE выполняются параллельно в узлах кластера
  - Этап SORT скрыт от пользователя «под капотом» платформы

Общая схема работы кластера Hadoop

MAP и REDUCE выполняются параллельно на узлах кластера, а SORT (shuffle) платформа делает сама — «под капотом»



один узел = один map или reduce; данные между ними переезжают по сети — это самая дорогая часть

Реальный коллекции документов постоянно меняются

- Например, веб-страницы постоянно создаются, изменяются и удаляются
- Пользователи ожидают от поисковой системы
  - свежесть – по запросу *выборы* надо находить сегодняшние документы
  - актуальность – по запросу *спартак зенит* надо в первую очередь находить документы про самый последний матч

Нужен механизм эффективного обновления индекса

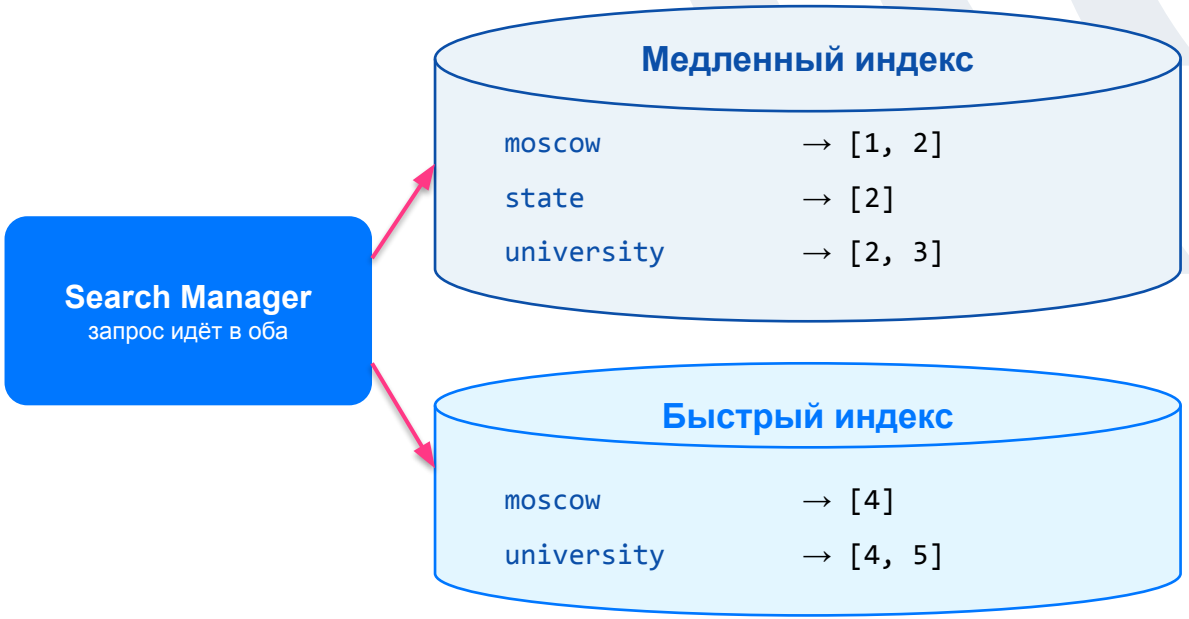
- Мы должны уметь:
  - Добавлять в индекс новые документы
  - Заменять старые версии документов на новые
  - Удалять «мертвые» документы (Ошибка 404, ...)
- Простейшее решение: просто перестроить индекс с нуля
  - В поиске VK большой индекс обновляется каждую неделю!
- Будем терять самые свежие документы

«Быстрый» и «медленный» индексы

Большой индекс перестраивается долго. Свежие документы кладём в маленький индекс в памяти, а запрос идём в оба.

Быстрый индекс

- Только свежие документы
- Целиком помещается в память
- Можно дописывать «на лету», без перестроения
- Маленький — пересечения по нему дешёвые

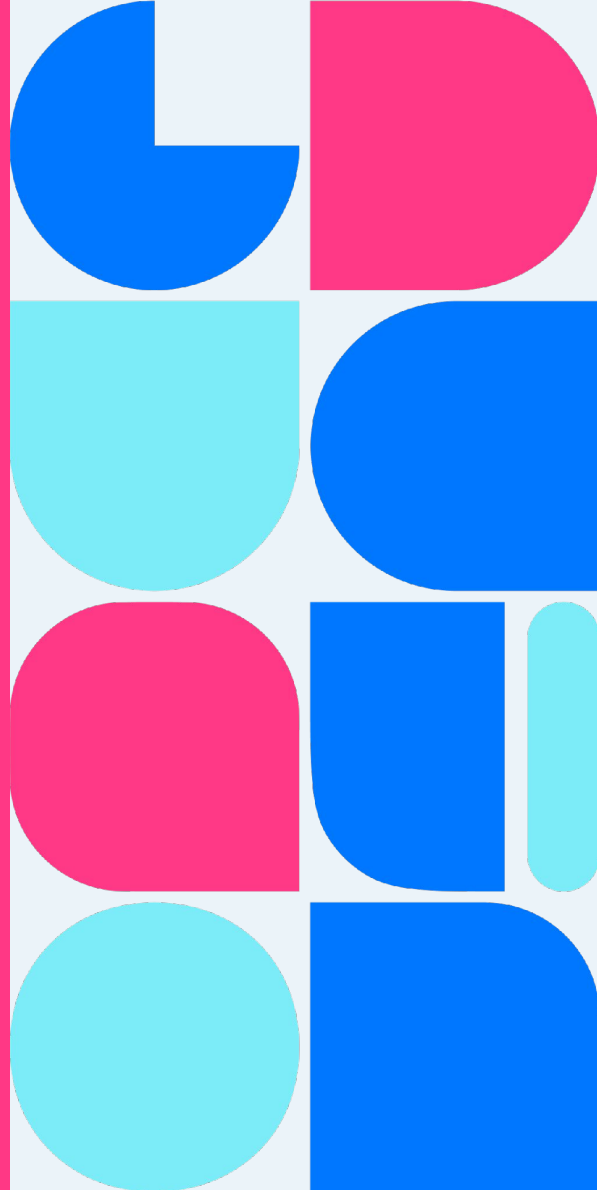


Что если документ нашёлся в обоих

Берём версию из быстрого — она свежее. Периодически быстрый индекс вливают в медленный и начинают новый

Сжатие индекса

Понять, зачем сжимают индекс, какими кодами и какой ценой



Зачем сжимать?

- Экономим место
  - Особенно если RAM
- Больше помещается в память
  - Быстрее передача данных
  - {Прочитать сжатое, распаковать} может быть быстрее чем {прочитать несжатое}
  - Больше можно закэшировать
- И мы говорим здесь не про gzip/rar/etc (десятки мб/сек), а действительно быстрые алгоритмы сжатия/распаковки, например LZO/LZ4 (гб/сек)

Сжатие с потерями

- Пример не из поиска: Jpeg
- Понижение капитализации, стоп-слова, морф. нормализация – может рассматриваться как сжатие с потерями
- Удаление координат для позиций, которые вряд ли будут вверху на ранжировании

Какие части индекса можно сжимать?

- Полезно сжимать
  - Словарь
  - Списки словопозиций
  - Списки вхождений в словопозициях
- Мы будем рассматривать только списки словопозиций в не-координатном индексе, т.е. отсортированные списки DocID
  - москва → [..., 283047, 283154, ...]
  - ...

Как сжимать списки DocID?

- Ключевая идея: храним промежутки (gaps) между последовательными DocID вместо самих DocID!
  - москва → [..., 283047, 283154, 283159, 283202, ...]
  - москва → [..., ..., 107, 5, 43, ...]
- Для частотных слов промежутки меньше
- Как следствие, промежутки распределены неравномерно: гораздо больше маленьких, чем больших
  - Маленькие промежутки можно кодировать меньшим числом байт (т.н. **кодирование с переменной длиной**) Какие есть идеи?

Кодирование с переменной длиной (variable byte encoding)

- Очень похоже на UTF-8
- 1-й бит каждого байта служит маркером конца кода
  - 1 – код заканчивается этим байтом, 0 – продолжается
- Последние 7-бит – полезная нагрузка, которая хранит части промежутка

DocIDs	824	829	215406
Gaps		5	214577
VarByte code	00000110 10111000	10000101	00001101 00001100 10110001

Свойства кодирования с переменной длиной

- Очень просто в реализации
- Уменьшает размер обратного индекса на 50%!
- Можем кодировать блоками больше или меньше байта (напр. 32 бита или 4 бита)
- Меньше размер блока → более экономно расходуется память
- На практике 1-байтовые коды почти оптимальны

Унарное кодирование

- Работает на битовом уровне
- Очень неэффективно само по себе, но используется в эффективном гамма-коде Элиаса
- Унарный код числа N: 111...(повторяется N раз)0
- Например: 5 → 111110

Гамма-код Элиаса

- Работает на битовом уровне
- Представляем промежуток в виде **длины** и **смещения**
- Для промежутка G:
  - Смещение Offset(G) – это G в двоичном коде без первой 1
  - Длина Length(G) – это длина смещения в унарном коде
  - Гамма-код Gamma(G) – конкатенация длины и смещения

Гамма-код Элиаса: пример

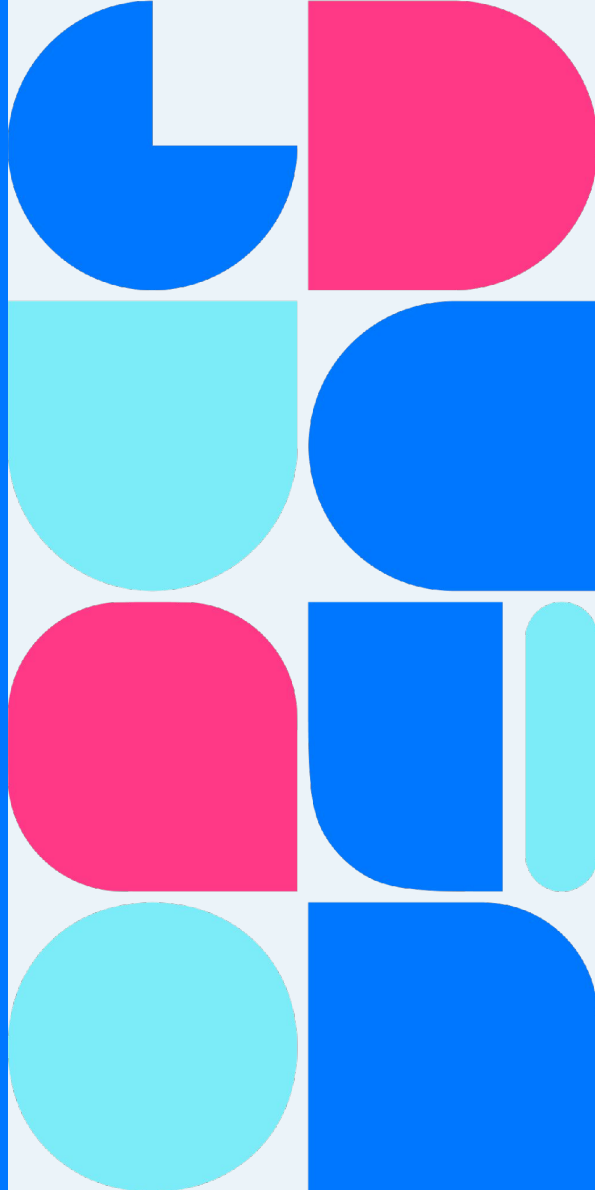
- Посчитаем гамма-код для G=13
  - Offset(13) = Offset(1101b) = 101
  - Length(13) = 3 бита = 1110
  - Gamma(13) = 1110101 (конкатенация длины и смещения)

Гамма-код Элиаса: свойства

- Можно показать, что гамма-код Элиаса почти оптимален
- Уменьшение размера индекса может достигать 75%!
- Для произвольной последовательности гамма-кодов всегда существует уникальный результат декодирования
  - При хранении и передаче кодов между ними не нужны разделители (если можем прочитать всю последовательность от начала)
- Универсальные: эффективны для произвольного распределения промежутков
- Нет каких-то дополнительных параметров, которые надо было бы настраивать под распределение

Оценка качества булевой модели

Научиться измерять качество булева поиска: точность, полнота и баланс между ними



Оценка качества булевой модели

- В булевой модели документ или удовлетворяет запросу, или нет
- По сути, это классификатор
- Каждому документу можно приписать метку класса:
  - 1 – релевантный
  - 0 – нерелевантный
- Большой дисбаланс классов: релевантных документов гораздо меньше чем нерелевантных

Матрица ошибок

- Результаты классификации удобно представить в виде матрицы ошибок
- y – реальное соответствие документа запросу
- h(x) – результат работы классификатора

	y = 1	y = 0
h(x) = 1	True Positive (TP)	False Positive (FP)
h(x) = 0	False Negative (FN)	True Negative (TN)

Точность и полнота

- Посчитаем TP, FP и FN
- Точность (precision) = TP / (TP + FP) — насколько релевантна показанная выдача
- Полнота (recall) = TP / (TP + FN) — сколько релевантного мы нашли
- Fβ-мера выражает баланс между точностью и полнотой:

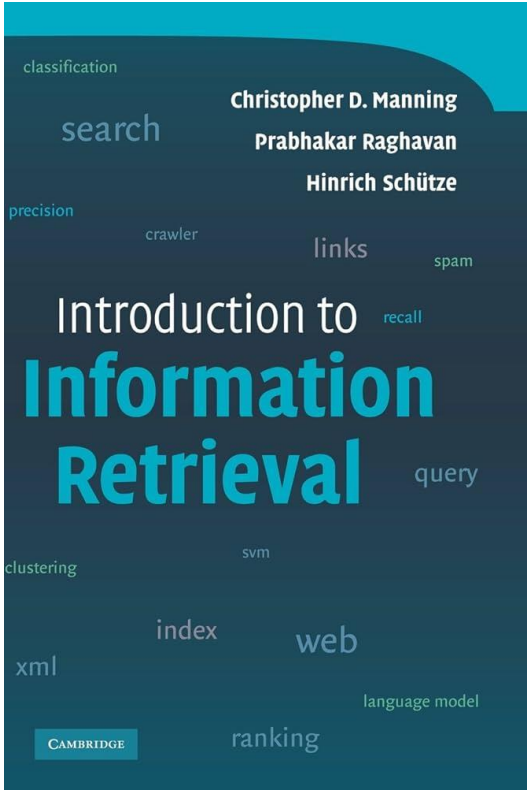
$$F_\beta = (1 + \beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}}$$

В домашнем задании используем F_1-меру (β = 1) — точность и полнота с равным весом

Повторение из лекции 1

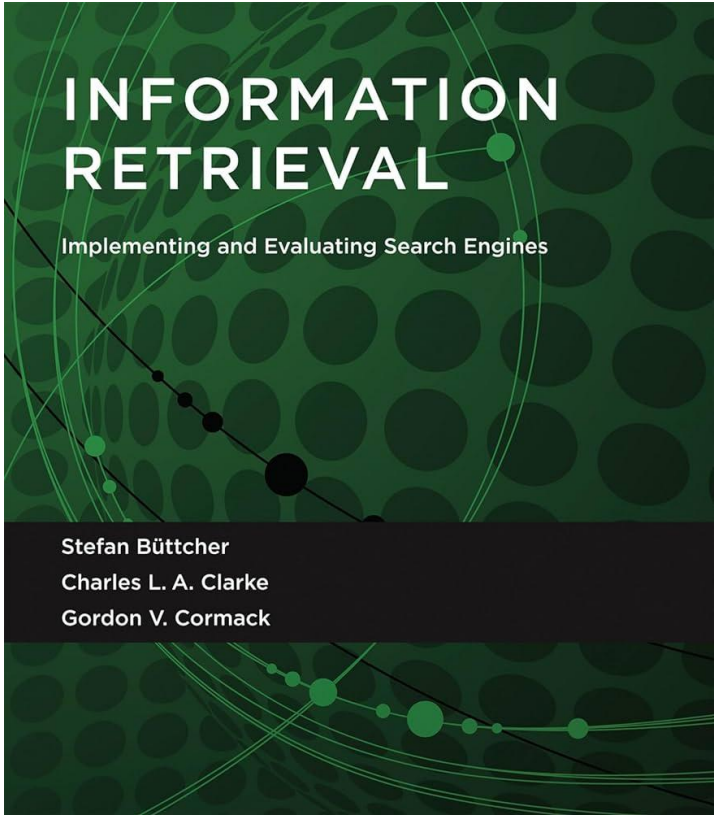
Что можно почитать

- **Introduction to Information Retrieval**
  Christopher D. Manning et al. (2008)
- Есть на русском!
- Английская версия доступна бесплатно
- Немного устарела
- Не очень строгое изложение



Лучший продвинутый учебник

- Лучший продвинутый учебник
- **Information Retrieval: Implementing and Evaluating Search Engines** Stefan Büttcher et al. (2016)
- Хорошо дополняет Маннинга



education

Домашнее задание

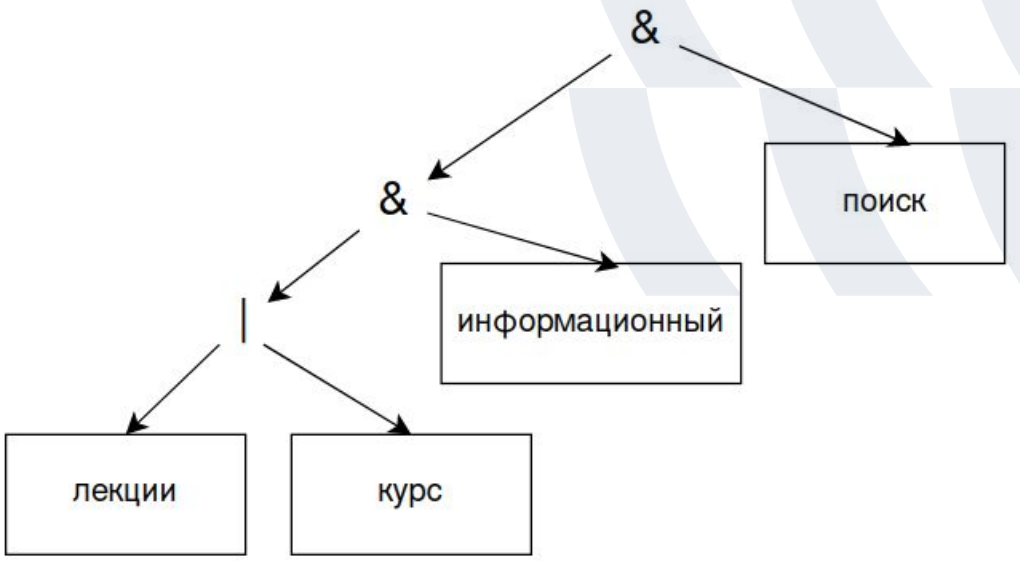


Домашнее задание

- Дана тестовая коллекция из небольшого количества документов и запросов
- Тексты запросов и документов уже лемматизированы
- Необходимо реализовать булеву модель и для каждого запроса найти множество удовлетворяющих ему документов
- Для сдачи задания F_1-мера результата должна попадать в заданный допустимый интервал

Дерево запроса

- Вспоминаем парсер арифметических выражений. Реализовывали такой?
- Например: *(лекции | курс) информационный поиск*



Дерево запроса: токенизация

- Все токены r'\w+|[\(\)&\|!]'
- Разбиваем на 3 класса:
  - Скобки
  - Операторы
  - Термы

Дерево запроса: алгоритм разбора

- Находим оператор с минимальным приоритетом
  - Наиболее внешний, наиболее правый
  - Запомним как token
- Если не нашли, значит результат = термин или None
- Иначе
  - token.left = рекурсивно слева
  - token.right = рекурсивно справа
  - результат = token

Исполнение дерева

- Вспоминаем исполнение дерева
- С каждым термином-листом связан свой список документов
- Документы «текут» вверх по дереву, в каждом узле происходит пересечение или слияние дочерних списков

Ссылки на домашнее задание

- Постройте обратный индекс для набора документов, сделайте поисковый движок и посмотрите насколько успешно находятся документы для разных запросов.
- Детали выложим в течение этой недели

Вопросы?



education

education

Семинар:
Библиотека whoosh

Перерыв



education

Спасибо
за внимание!

education